Eventgen Manager 2.0

A complete, step-by-step tutorial

Configure, preview, and generate realistic test data in Splunk from the browser, using the app's built-in engine, with no hand-editing of eventgen.conf.

App author: Kamal Singh Bisht • Version 2.0.0 • Apache License 2.0

Updated November 2026

Contents

1. What Eventgen Manager does 3

2. Before you start 4

3. Check that the engine is ready 5

4. A tour of the Configurations page 6

5. Start your first data in one click 8

6. The configuration editor, tab by tab 10

7. Build your own data from scratch 24

8. Clone a configuration 31

9. Monitor throughput 33

10. Control the engine 35

11. Read the warning signs 37

12. Troubleshooting 40

13. Reference 41

1. What Eventgen Manager does

Splunk demos, detection testing, dashboard building, and load tests all need data that looks real. The usual tool for that is eventgen, which reads a file called `eventgen.conf`. That file is powerful but unforgiving: one stanza per sample file, regular expressions for every value you want to change, JSON for rate curves, and a dozen output modes.

Eventgen Manager puts a guided editor in front of all of that. You pick a sample file, set how often and how many events to send, choose what to replace in each line, and watch a live preview of the events before a single one reaches an index. Since version 2.0 the app also generates the events itself with a built-in engine, so SA-Eventgen is no longer needed.

The app has five pages, reached from the dark bar at the top of every screen:

Page	What you use it for
Configurations (home page)	See every eventgen configuration on the instance, start and pause them, watch live counts, and open the editor to create, edit, or clone.
Sample data	Upload, write, view, and delete the sample files that configurations generate from.
Throughput	Events per second, projected daily volume, failed batches, configured against measured rates, and engine warnings.
Generator control	Health checks, start and stop the engine, and tune how quickly it picks up changes.
Tutorial	Opens this guide inside Splunk, with a quick-start checklist.

A typical session looks like this:

1. Check **Generator control** shows everything green.
2. Add or pick a sample file on **Sample data**.
3. Create or edit a configuration, watching the **Live preview**.
4. Press **Send test events** to confirm the data lands where you expect.
5. Tick **Generate events with this configuration** and save. The engine starts it within about ten seconds.
6. Watch the **Generated** column and the **Throughput** page.

2. Before you start

Requirements

- Splunk Enterprise 9.0 or later, or Splunk Cloud.
- The **admin** or **sc_admin** role to create or change configurations and sample files. Users with the `list_inputs` capability can look but not change anything.
- Nothing else. SA-Eventgen is not required (see Section 13 if you already have it).

Install the app

1. Install from Splunkbase, or extract the package into `$SPLUNK_HOME/etc/apps/`.
2. Restart Splunk.
3. Open **Eventgen Manager** from the **Apps** menu. You land on the Configurations page.

> **Good to know.** The engine ships turned on, but all four starter configurations ship turned off. Nothing is generated until you start something, so installing the app is safe on any instance.

What ships with the app

Starter configuration	What it produces	Default schedule
`access_combined.sample`	Web server access logs with random client IPs, users from a file, weighted HTTP status codes, and an hour-of-day traffic curve.	20 events every 10 s
`linux_secure.sample`	Linux authentication (sshd, sudo) logs with host names from a list.	6 events every 30 s
`app_json.sample`	Structured JSON application logs in UTC, with trace IDs and latency that rises at busy hours.	25 events every 5 s
`firewall_traffic.sample`	Cisco ASA style firewall traffic with connection IDs that never repeat.	12 events every 15 s

Two helper files, `users.sample` and `hosts.sample`, supply values for the starters. Every address in the bundled samples comes from ranges reserved for documentation, so none of it points at a real host or person.

Open this tutorial from inside Splunk

This guide ships with the app. Click **Tutorial** in the app's top bar, then press **Open the tutorial**. The PDF opens in a new browser tab, where you can read, search, or download it. The page also lists what the guide covers and a five-step quick start.

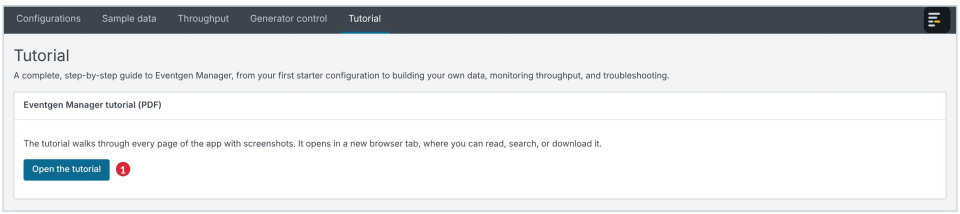


Figure 2.1 — The Tutorial page in the app.

3. Check that the engine is ready

Before building anything, open **Generator control** from the top bar. This page answers one question: is everything in place for events to be generated?

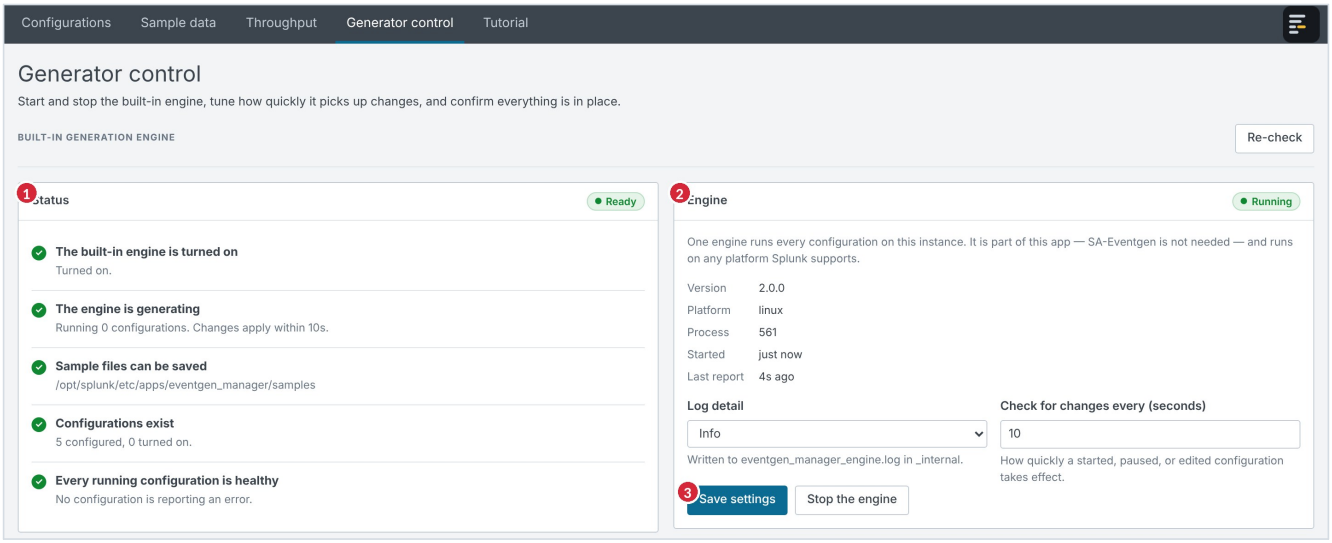


Figure 3.1 — Generator control on a fresh install. Every check is green and the engine is running.

1. **Status** lists five health checks. A green tick means that part is fine; an amber mark explains what is wrong and what to do. The badge at the top reads **Ready** or **Needs attention**.
2. **Engine** shows the built-in engine's version, platform, process ID, when it started, and when it last reported in.
3. **Save settings** applies changes to the log level and the reload interval. **Stop the engine** pauses all generation at once.

The five checks mean:

Check	What it confirms
The built-in engine is turned on	The `eventgen_manager_engine://default` input is enabled.
The engine is generating	The engine has reported in during the last 30 seconds, and how many configurations it is running.
Sample files can be saved	Splunk can write to the app's samples folder (path shown).
Configurations exist	How many configurations exist and how many are turned on.
Every running configuration is healthy	No running configuration is reporting an error.

If anything is amber, the detail line tells you the fix. Section 12 walks through the common cases.

4. A tour of the Configurations page

Configurations is the home page. It lists every eventgen stanza on the instance, from every app, not only this one.

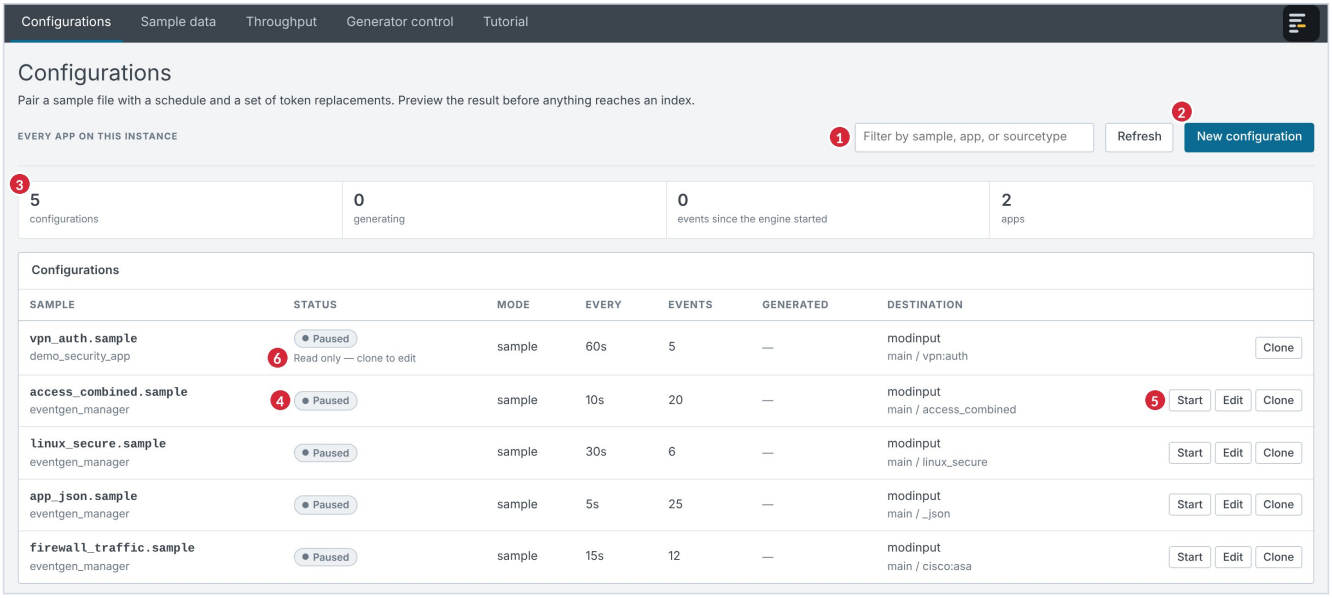


Figure 4.1 — The Configurations page right after install: four starters, all paused, plus one read-only stanza from another app.

1	**Filter** narrows the list by sample name, app, sourcetype, or index as you type. **Refresh** re-reads everything.
2	**New configuration** opens an empty editor.
3	Summary tiles: total configurations, how many are generating, events produced since the engine started, and the number of apps (this tile turns into a red **failing** count if anything errors).
4	**Status** of each configuration (table below).
5	Row actions: **Start**/**Pause**, **Edit**, **Clone**, and **Delete** (only for configurations you created).
6	Stanzas owned by other apps are **Read only**. You can clone them but not edit them.

Columns

Column	Meaning
Sample	The stanza name (usually the sample file) and the app it lives in.
Mode	**sample** repeats the file on a schedule; **replay** plays it back at its original pace.
Every	The interval between batches, in seconds.
Events	Events per batch, a GB/day target, or *whole file* when the count is 0.
Generated	Live count from the engine and when the last batch went out.
Destination	Output mode, and index / sourcetype for outputs that index into Splunk.

Status values

Status	Meaning
Paused	Turned off. Nothing is generated.
Starting	Turned on; the engine has not picked it up yet (up to one reload interval).
Generating	Running normally.
Backfilling	Catching up on a past time window you asked for with *Backfill from*.
Finished	Reached its *Stop after* limit, or a replay ran out of events.
Error	Failing. The reason appears right under the badge.
Idle / Standby	Turned on, but the engine itself is off (Idle) or waiting because SA-Eventgen is running (Standby).

> **Tip.** Counts refresh by themselves every five seconds while the page is open. You never need to restart Splunk or the engine after a change.

5. Start your first data in one click

The quickest way to see the app work is to start a starter as it is. In this example we start **linux_secure.sample** and **app_json.sample**.

1. On **Configurations**, press **Start** on the row you want.
2. A green message confirms it and says the engine picks it up within ten seconds.
3. Wait a few seconds. The status changes to **Generating** and the **Generated** column starts counting.

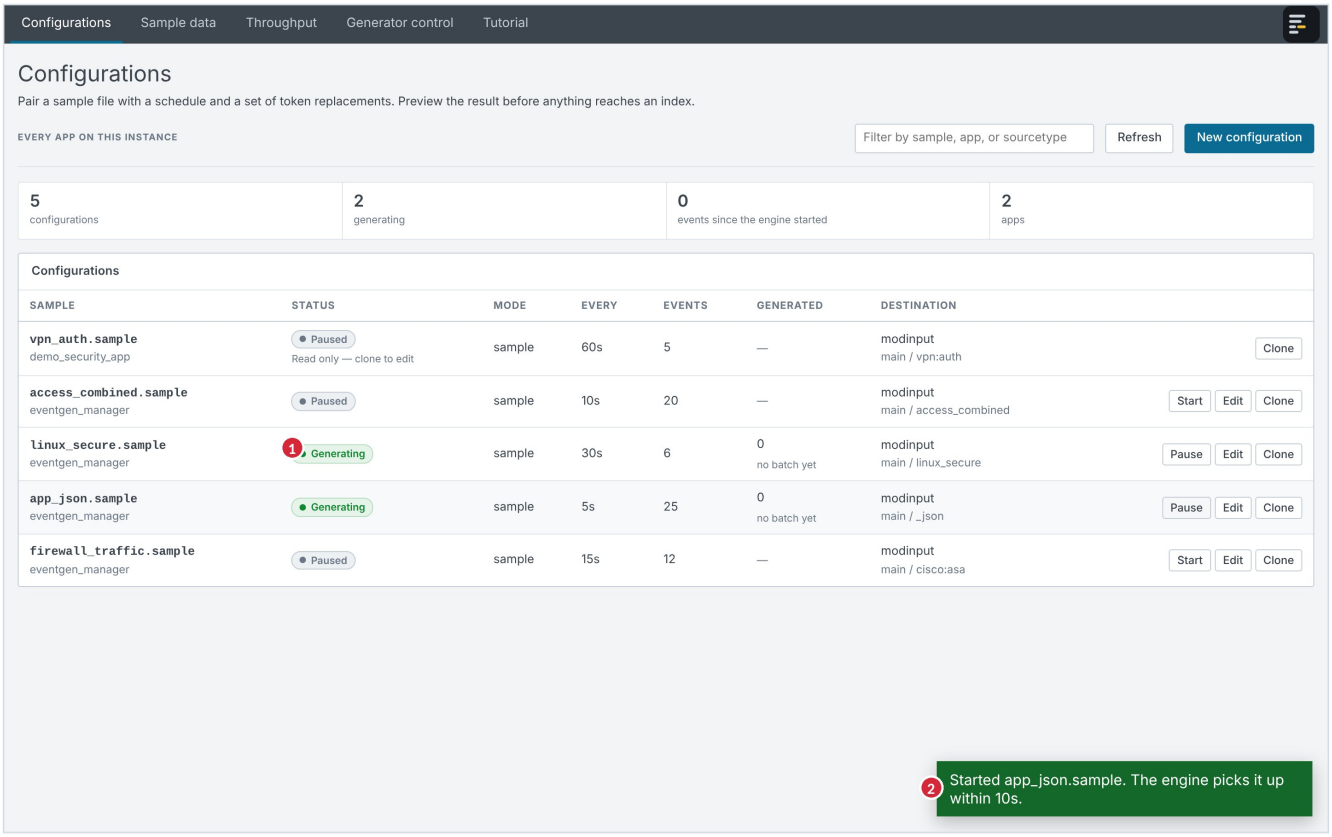


Figure 5.1 — After pressing Start (1). The confirmation message (2) tells you when the change takes effect.

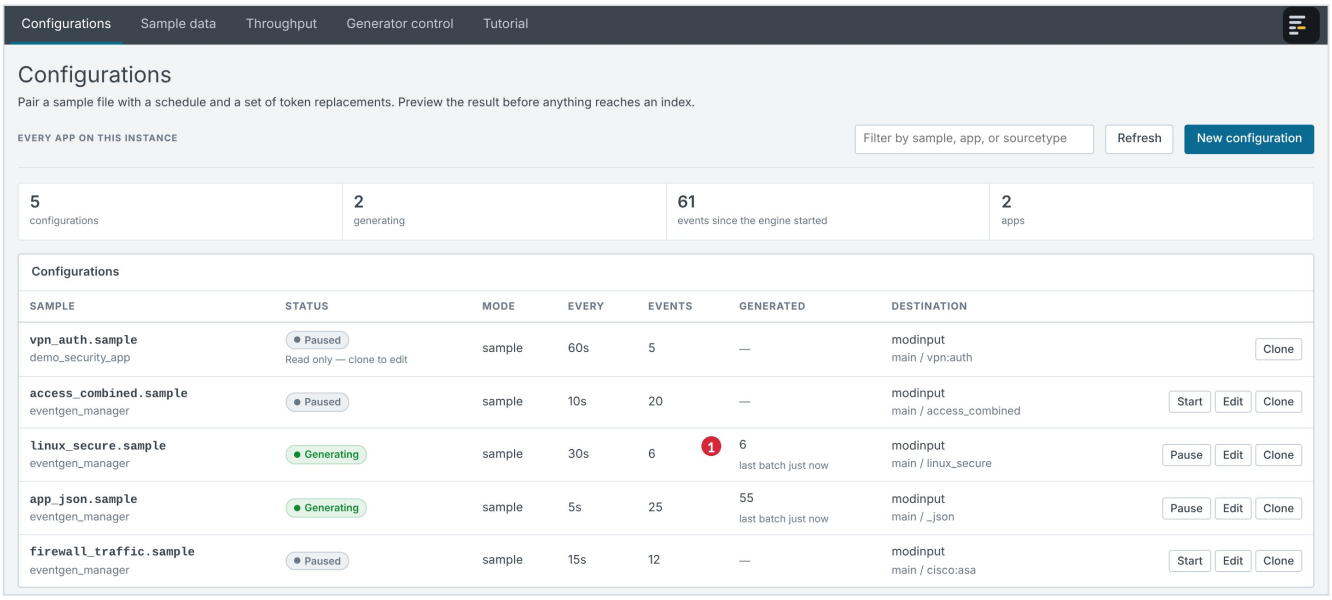


Figure 5.2 — A few seconds later. Generated counts (1) and "last batch just now" show the engine is producing events.

To check the data in Splunk, run a search such as `index=main sourcetype=linux_secure earliest=-15m`. To stop, press **Pause** on the same row.

> **Good to know.** Starting a starter directly writes `disabled = false` to `local/eventgen.conf`. If you later delete the app folder to reinstall, `local/` goes with it and the starters fall back to paused. For anything you want to keep, clone or build your own configuration (Sections 7 and 8).

6. The configuration editor, tab by tab

Press **Edit** on any configuration (here, **access_combined.sample**) to open the editor. The same editor is used for new configurations and clones. The left side holds the settings, split into tabs; the right side shows the live preview.

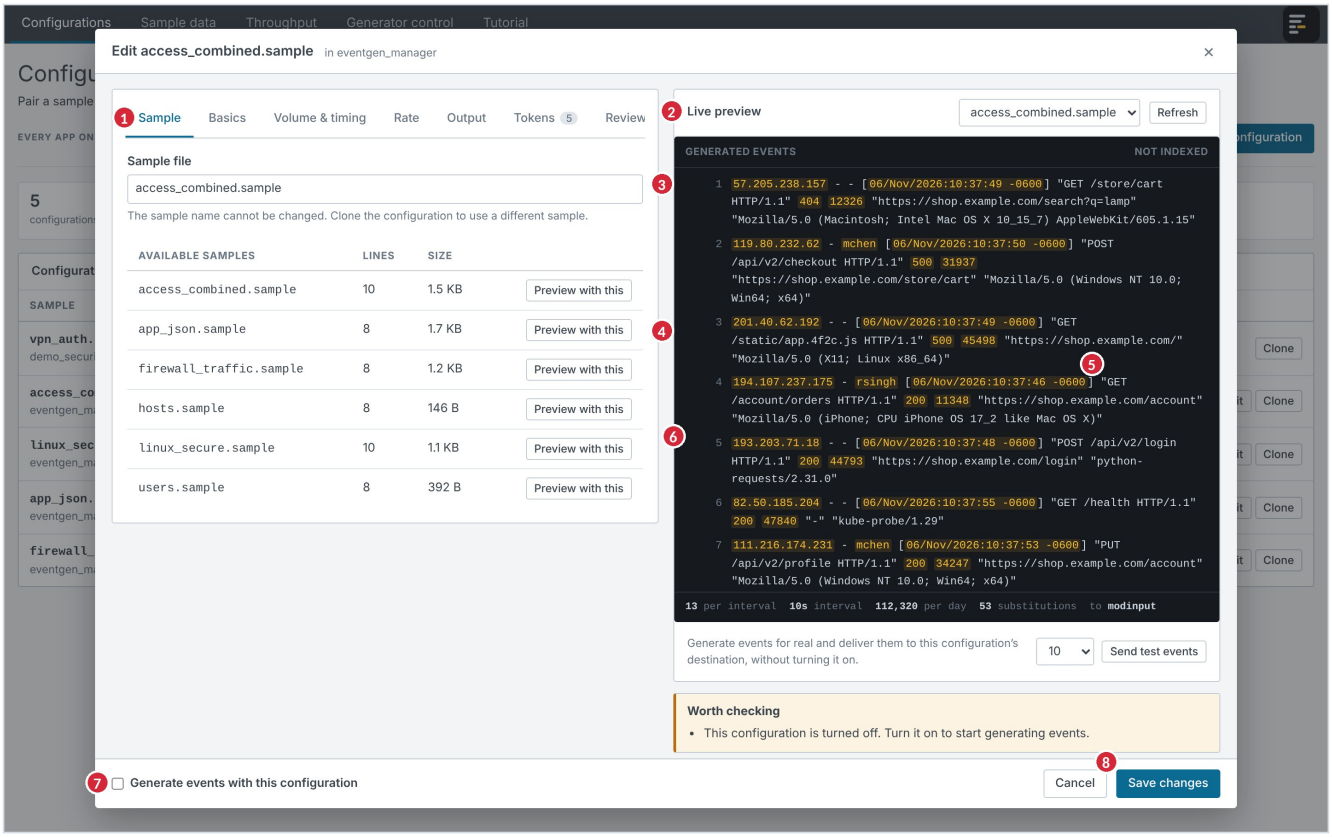


Figure 6.1 — The editor, opened on the Sample tab.

1	Tabs: Sample, Basics, Volume & timing, Rate, Output, Tokens (with a count), and Review.
2	**Live preview**. The drop-down lets you preview with a different sample file; **Refresh** draws a fresh set of events.
3	Generated events. Every value a token replaced is highlighted in yellow.
4	Summary line: events per interval, interval, projected events per day, number of substitutions, and the output.
5	**Send test events** generates a few events for real and delivers them, without turning the configuration on.
6	**Worth checking** lists anything that looks wrong, such as a token that matches nothing or an unsupported setting.
7	**Generate events with this configuration**: the on/off switch, saved with the rest.
8	**Save changes** (or **Create configuration** for a new one). **Cancel** or Esc closes without saving.

> **Tip.** Notice the preview shows **2 per interval** even though the count is 20. This configuration has an hour-of-day curve, and the screenshot was taken around 1 a.m., when the curve is set to 0.1. The preview always applies your rate curves to the count.

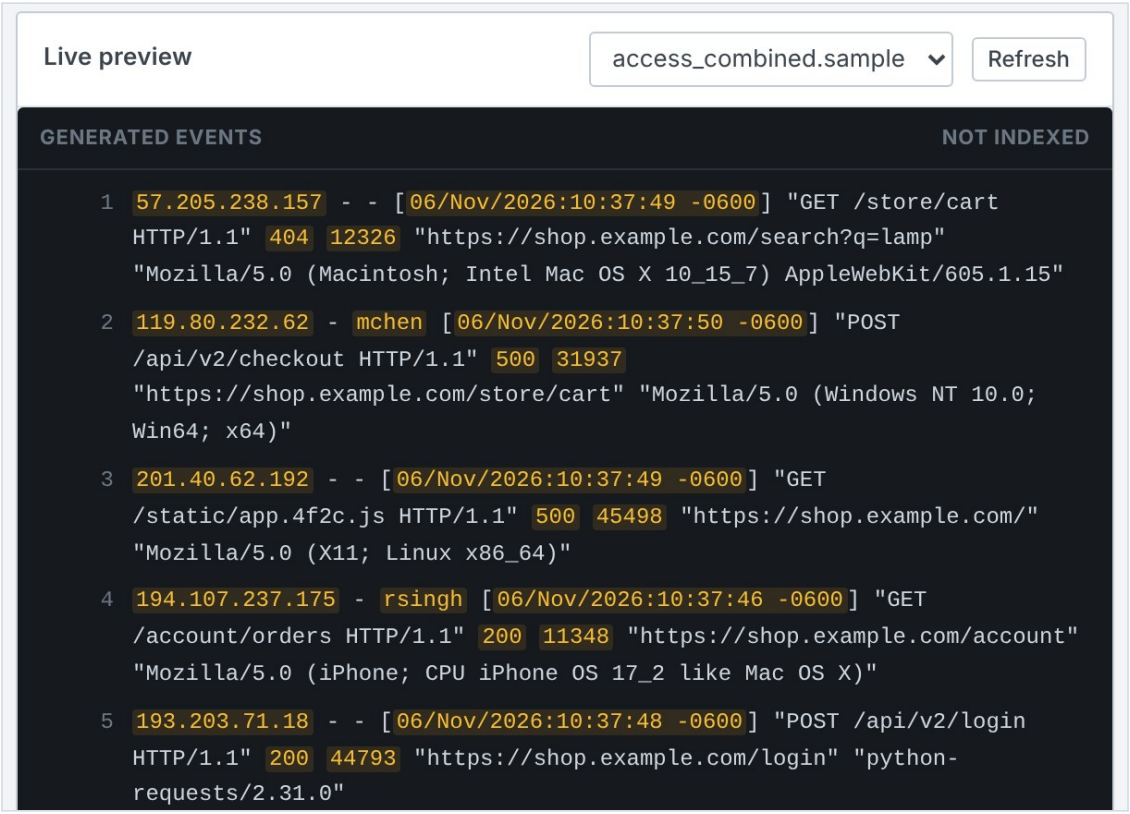


Figure 6.2 — Close-up of the preview. Yellow marks each substituted value: IP address, user, timestamp, status code, and response size.

6.1 Sample tab

Shows which sample file this configuration uses and lists the files available in the app. For an existing configuration the name is fixed; clone it to use a different sample. For a new one, type a file name or press **Use this** next to a file. The name can also be a regular expression, which applies the same settings to every matching file.

If you or someone else added settings by hand that the editor has no field for, a blue note on this tab lists them as **Settings kept as they are**. They are written back untouched when you save.

6.2 Basics tab

Field (setting)	What it does
Sample directory (`sampleDir`)	Where the sample file lives. Leave empty for the app's `samples` folder. An absolute path here is never touched by an app upgrade.
Generation mode (`mode`)	**Sample** emits a batch every interval, forever. **Replay** plays the file back at its original pace, using the gaps between its timestamps.
Sample format (`sampletype`)	**Raw**: one event per line. **CSV**: a header row plus a `_raw` column; CSV rows can set their own host, source, sourcetype, and index.
Detect timestamps automatically (`autotimestamp`)	Finds the first ISO 8601, web log, syslog, or US-style timestamp in each event and replaces it, keeping its format, so you do not need a timestamp token.
Timestamp to write (`autotimestampTime`)	Which time the detected timestamp gets: **Generated time** (within earliest and latest, the default) or **Current time** (the moment each event is sent). Only used when Detect timestamps automatically is ticked. See 6.2.1.

6.2.1 Automatic timestamps and Timestamp to write

Ticking **Detect timestamps automatically** is the fastest way to get fresh timestamps: the app spots the timestamp already in each sample line and swaps in a new one in the same format. No token or regular expression needed. **Timestamp to write** then decides *which* time goes in.

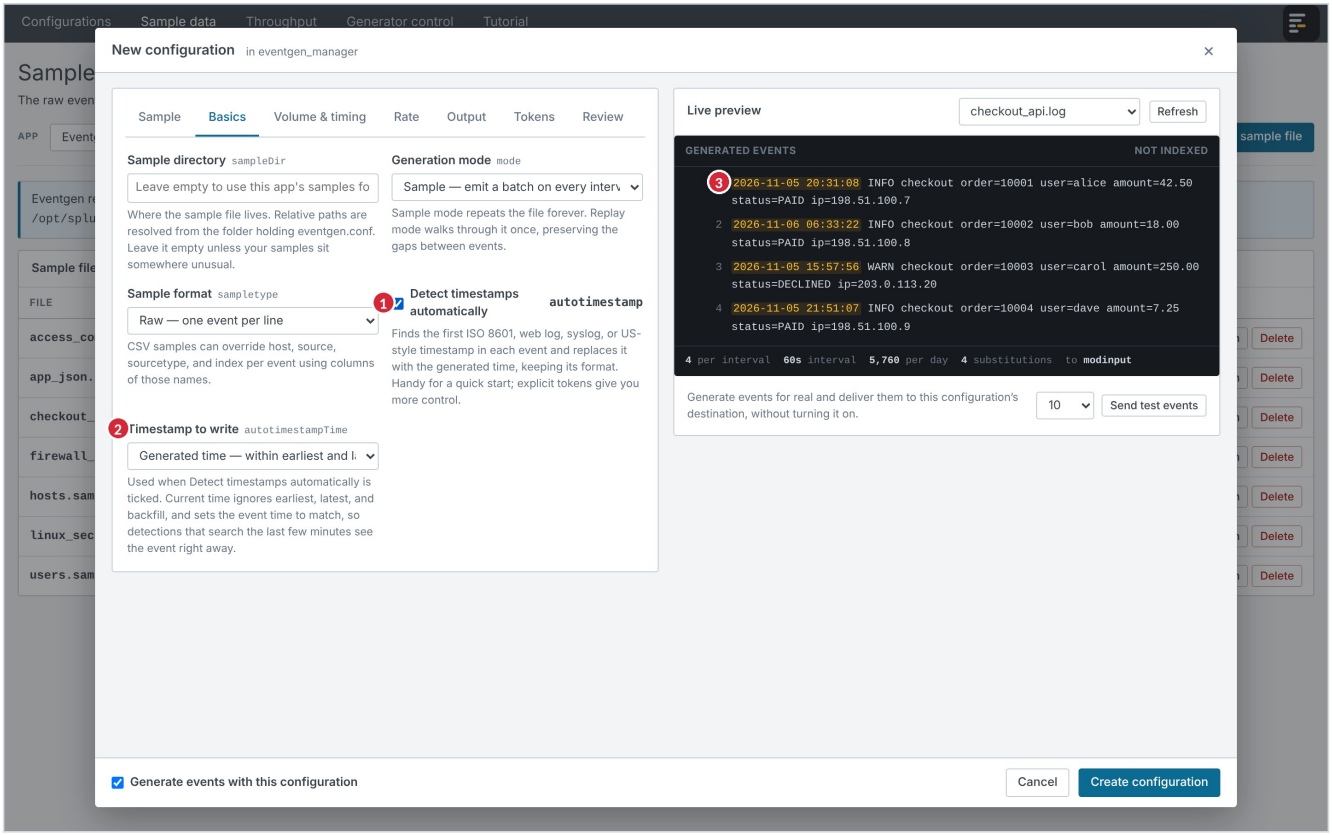


Figure 6.3 — The Basics tab with automatic timestamps on. Earliest is set to -24h, so Generated time spreads events across the last day.

1 **Detect timestamps automatically**: turn this on first.

2 **Timestamp to write**: Generated time or Current time.

3 The preview highlights the replaced timestamp in each event.

Generated time (`generated`)

GENERATED EVENTS — NOT INDEXED

```
1 2026-11-05 20:31:08 INFO checkout order=10001 user=alice amount=42.50
  status=PAID ip=198.51.100.7
2 2026-11-06 06:33:22 INFO checkout order=10002 user=bob amount=18.00
  status=PAID ip=198.51.100.8
3 2026-11-05 15:57:56 WARN checkout order=10003 user=carol amount=250.00
  status=DECLINED ip=203.0.113.20
4 2026-11-05 21:51:07 INFO checkout order=10004 user=dave amount=7.25
  status=PAID ip=198.51.100.9
```

Times are picked between earliest (-24h) and latest (now): yesterday morning, mid-morning, evening…

Current time (`now`)

GENERATED EVENTS — NOT INDEXED

```
1 2026-11-06 10:39:54 INFO checkout order=10001 user=alice amount=42.50
  status=PAID ip=198.51.100.7
2 2026-11-06 10:39:54 INFO checkout order=10002 user=bob amount=18.00
  status=PAID ip=198.51.100.8
3 2026-11-06 10:39:54 WARN checkout order=10003 user=carol amount=250.00
  status=DECLINED ip=203.0.113.20
4 2026-11-06 10:39:54 INFO checkout order=10004 user=dave amount=7.25
  status=PAID ip=198.51.100.9
```

Every event is stamped with the moment it is built, whatever earliest and latest say.

Figure 6.4 — The same sample and the same -24h window, with each setting.

	Generated time (default)	Current time
Timestamp in the event	Random time between earliest and latest (or across the backfill window)	The moment the event is built
Earliest, latest, backfill	Used	Ignored for this timestamp
Event time in Splunk (`_time`)	The generated time	Set to the same current time, so the text and `_time` always agree
Best for	History: filling dashboards, trend charts, and searches over hours or days	Live testing: alerts and correlation searches that look at the last few minutes

To set it up:

1. Open the configuration and go to **Basics**.
2. Tick **Detect timestamps automatically**.
3. Under **Timestamp to write**, choose **Current time — the moment each event is sent**.
4. Check the preview: every highlighted timestamp should read the current time.
5. Save. The Review tab shows the two settings written to the stanza:

```
[checkout_api.log]
autotimestamp = true
autotimestampTime = now
disabled = false
earliest = -24h
latest = now
```

Figure 6.5 — What gets saved to eventgen.conf.

> **Tip.** Choose Current time when a detection searches a short window, for example the last 5 minutes. With Generated time and a wide window, many events land outside that window and the alert never fires. With Current time, every event is in range the moment it is indexed.

> **Good to know.** Timestamp to write only affects the timestamp that automatic detection finds. If you leave Detect timestamps automatically unticked, the setting does nothing, and your timestamp tokens follow earliest and latest as usual. Avoid using automatic detection and a timestamp token on the same timestamp; pick one.

6.3 Volume & timing tab

Controls how many events are produced, how often, and which timestamps they carry. Every field shows the `eventgen.conf` setting it writes next to its label, so what you learn here carries over to the file format.

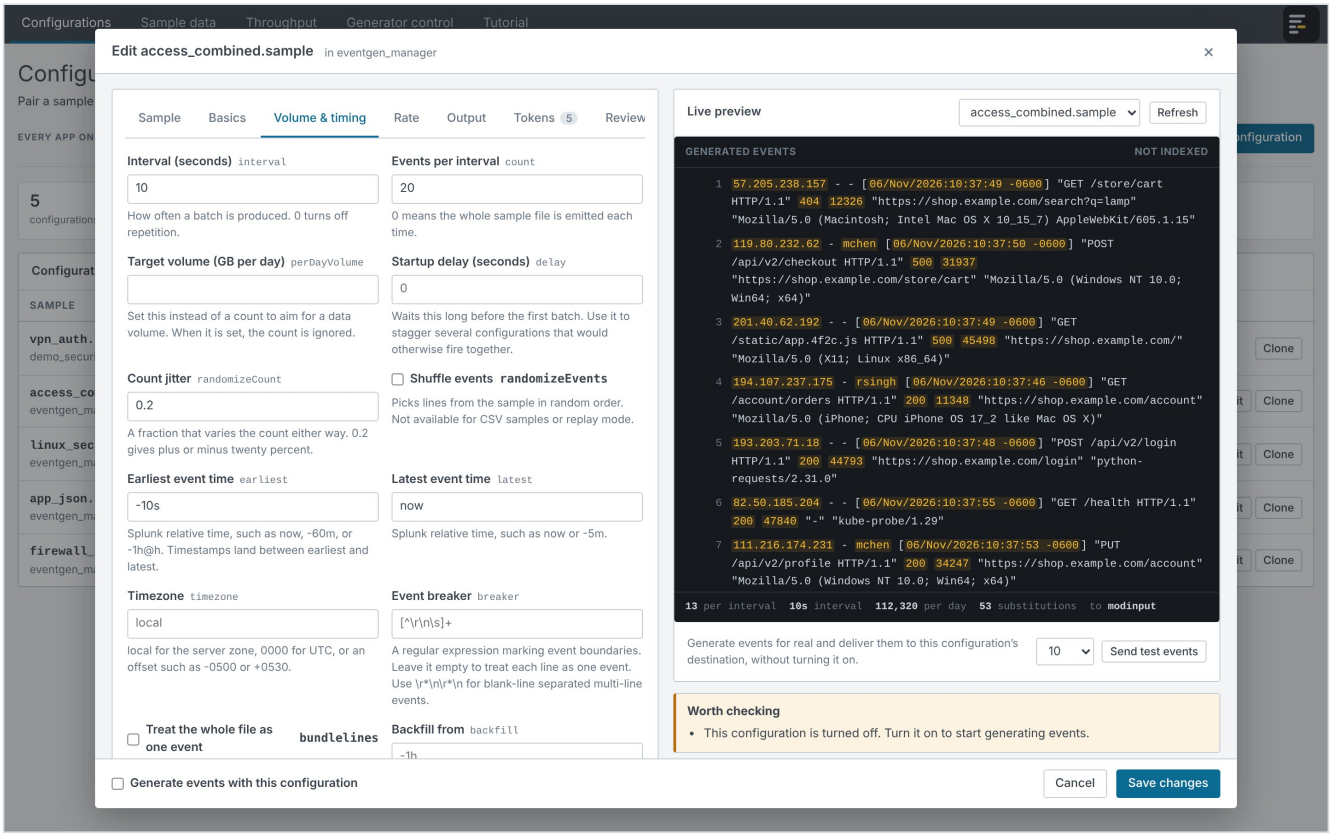


Figure 6.6 — Volume & timing. Fields appear only when they apply to the chosen mode.

Field (setting)	What it does
Interval (`interval`)	Seconds between batches. 0 turns off repetition.
Events per interval (`count`)	Events in each batch. 0 means the whole sample file every time.
Target volume (`perDayVolume`)	Aim for a daily volume in GB instead of a count. When set, the count is ignored.
Startup delay (`delay`)	Wait this long before the first batch, to stagger configurations.
Count jitter (`randomizeCount`)	Vary the count either way. 0.2 means plus or minus 20 percent.
Shuffle events (`randomizeEvents`)	Pick lines from the sample in random order.
Earliest / Latest event time (`earliest`, `latest`)	The time window that generated timestamps fall in, in Splunk relative time (for example `-60s` and `now`).
Timezone (`timezone`)	`local`, `0000` for UTC, or an offset such as `-0500`.
Event breaker / whole file (`breaker`, `bundlelines`)	How to split the sample into events, for multi-line samples.
Backfill from (`backfill`)	Generate a past window first, such as `-1h`, then carry on in real time.
Stop after (`end`)	Stop after a number of intervals or at a point in time.
Replay speed / Timestamp field (`timeMultiple`, `timeField`)	Replay mode only: speed up or slow down playback, and which CSV column holds the time.

6.4 Rate tab

Rate curves make traffic rise and fall like real traffic. Instead of typing JSON, you fill in one box per hour (0 to 23) and per day (Sunday to Saturday). A value of 1 means the full count, 0.5 halves it. All curves that match the current time are multiplied together.

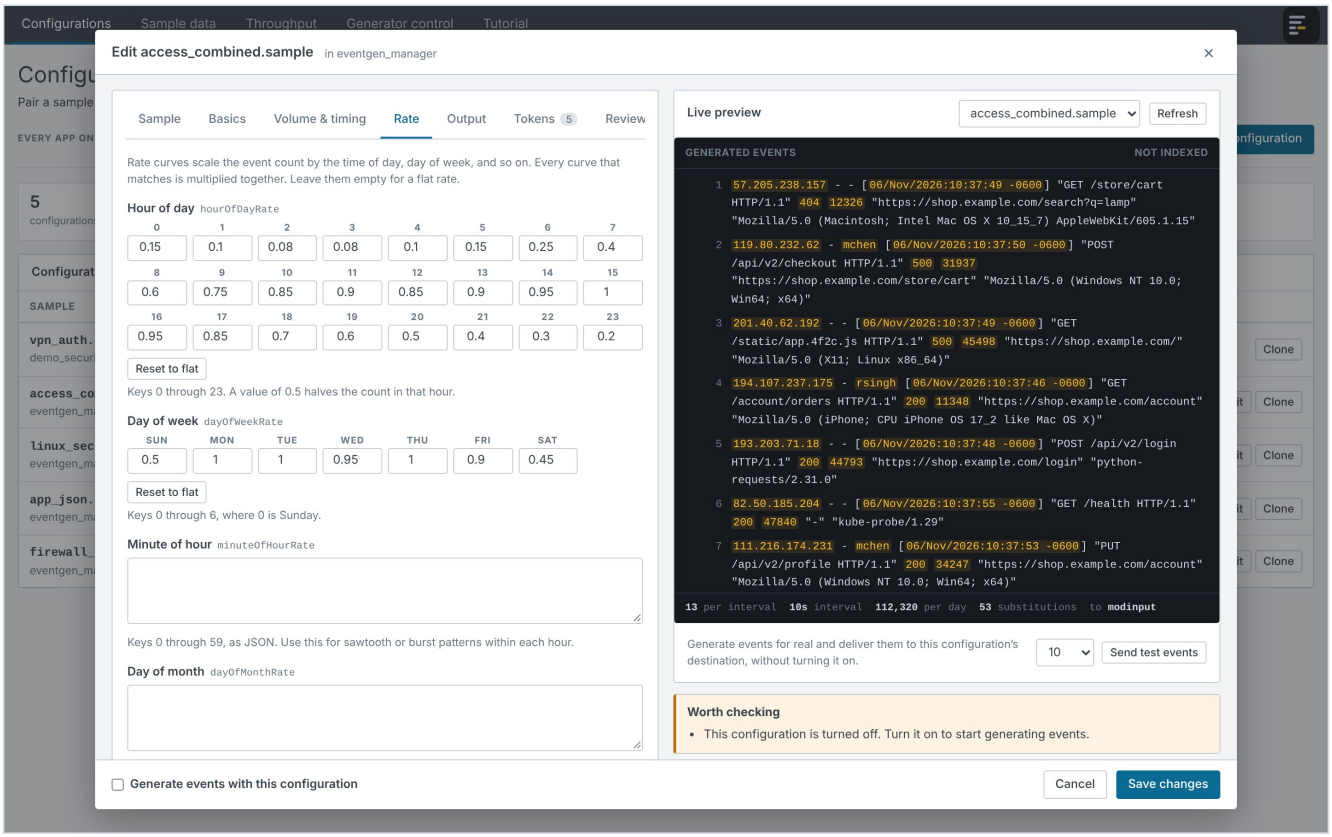


*Figure 6.7 — Hour-of-day and day-of-week curves as editable buckets. **Reset to flat** clears a curve.*

Minute-of-hour, day-of-month, and month-of-year curves are also supported, entered as JSON (keys 0–59, 1–31, and 1–12). The **rated** token type (Section 6.6) uses these same curves to scale numbers, for example latency that climbs at busy hours.

6.5 Output tab

Decides where generated events go. The form changes to match your choice: pick HTTP Event Collector and you see collector endpoints; pick a file and you see a path and rotation settings.

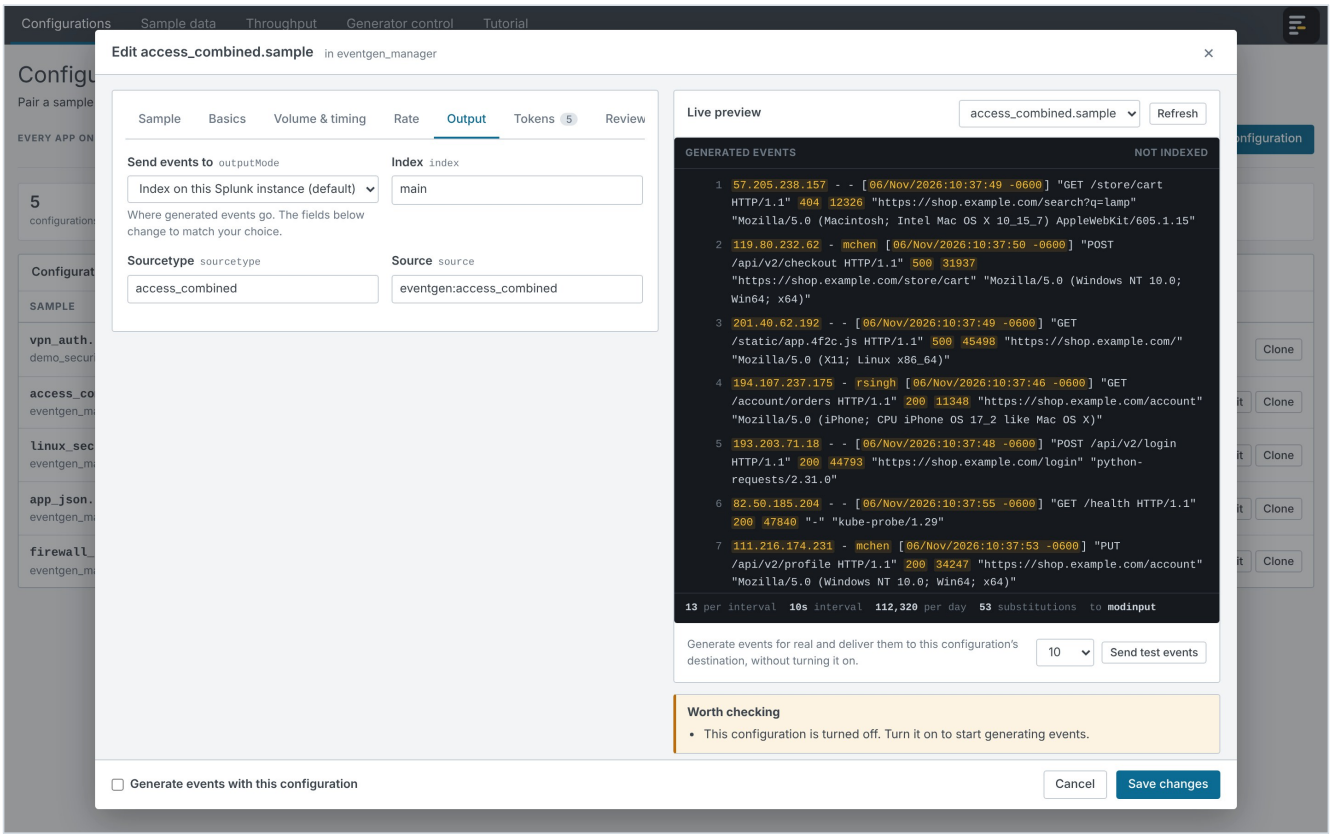


Figure 6.8 — The default output indexes events on this Splunk instance.

Send events to	Use it when
Index on this Splunk instance (`modinput`)	The default. Events are indexed locally with the index, sourcetype, source, and host you set.
Splunk REST (`splunkstream`)	Sending to another Splunk instance through its management port.
HTTP Event Collector (`httpevent`)	Sending to a HEC endpoint, here or elsewhere.
Write to a file (`file`)	Producing a log file for a forwarder or another tool to pick up.
Spool directory (`spool`)	Dropping files into a folder Splunk monitors.
Syslog over TCP (`syslogout`)	Feeding syslog-ng, rsyslog, or a Splunk TCP input.
Raw TCP (`tcpout`)	Sending plain lines to a TCP port.
Standard output / Discard (`stdout`, `devnull`)	Troubleshooting, or measuring generation speed alone.

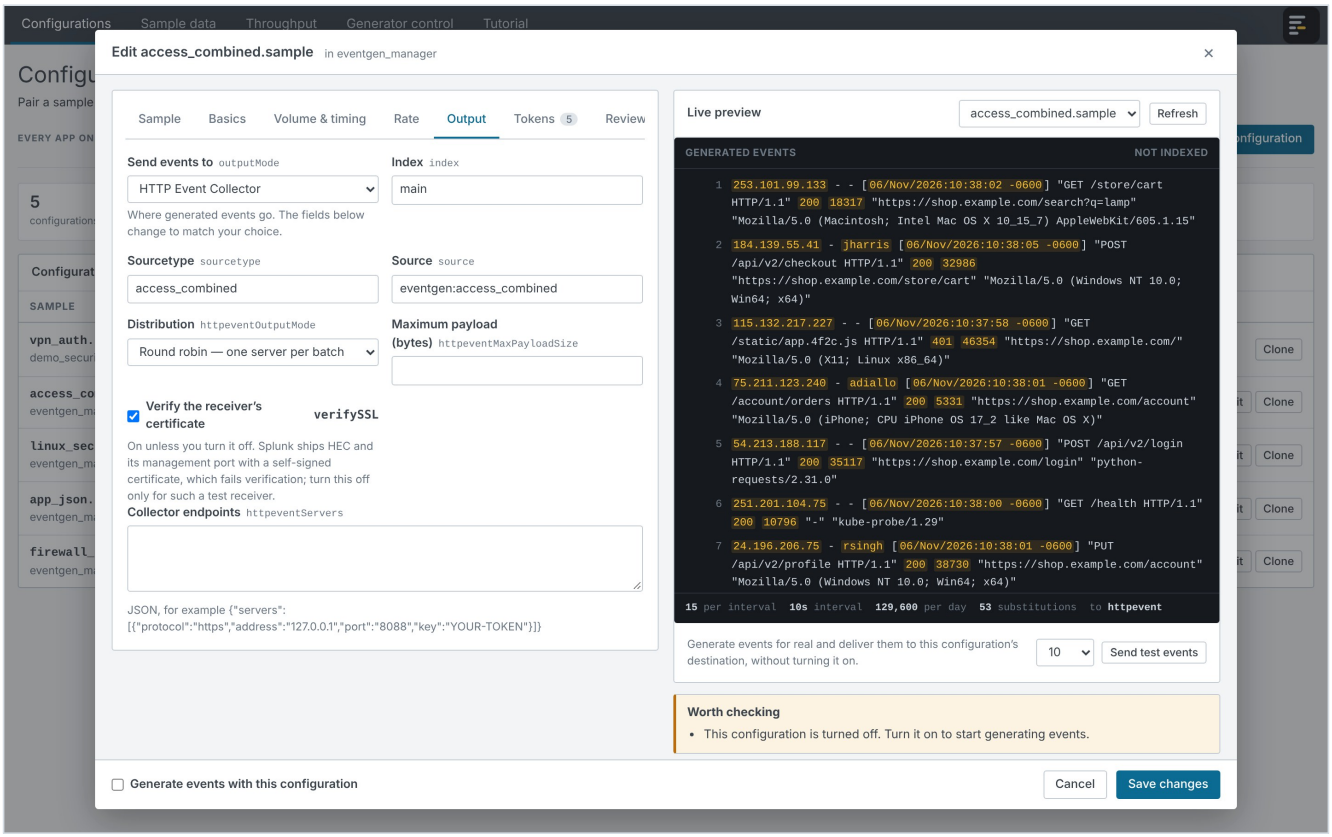


Figure 6.9 — Choosing HTTP Event Collector shows the collector endpoints, distribution, payload size, and certificate check.

For HEC, enter the endpoints as JSON, for example:

```
{"servers": [{"protocol": "https", "address": "127.0.0.1",
              "port": "8088", "key": "YOUR-HEC-TOKEN"}]}
```

> **Good to know.** Splunk's own HEC and management ports use a self-signed certificate by default, which fails verification. Untick **Verify the receiver's certificate** only for such a test receiver. UDP is not available (Splunk Cloud and app certification do not allow it), so syslog goes over TCP.

Sending data to your indexers (splunktcp)

There is no separate S2S or `splunktcp` output, and you do not need one. With **Index on this Splunk instance** (`modinput`), generated events enter the instance's normal data pipeline like any other input. If the instance forwards data, Splunk sends the events to your indexers over `splunktcp` (usually port 9997) by itself, with its own TLS and load balancing.

Set up forwarding in `system/local/outputs.conf` on the instance running the app:

```
# $SPLUNK_HOME/etc/system/local/outputs.conf
[tcpout]
defaultGroup = primary_indexers
# Forward only, keep no local copy (heavy forwarder)
indexAndForward = false

[tcpout:primary_indexers]
server = idx1.example.com:9997, idx2.example.com:9997
```

- Keep each configuration's output on `modinput`. The index, sourcetype, source, and host you set travel with the events.
- Create the index on the indexers, not only on the instance running the app. Otherwise events are dropped or land in a default index.
- To forward only the generated data, without changing what else the instance forwards, set `_TCP_ROUTING` on the engine's input. This applies to every configuration the engine runs. Try it on a non-production instance first.

```
# $SPLUNK_HOME/etc/apps/eventgen_manager/local/inputs.conf
[eventgen_manager_engine://default]
_TCP_ROUTING = primary_indexers
```

> **Tip. Send test events** follows the same path: the events are indexed locally and forwarded by the same rules. If the app cannot run on a forwarder, or you are sending to Splunk Cloud, use **HTTP Event Collector** instead. The **Raw TCP** output sends plain lines, so point it at a plain `[tcp://...]` input, never at a `splunktcp` port.

6.6 Tokens tab

Tokens are what make generated data look alive. Each token finds text in the sample with a regular expression and swaps it for something generated. They run in order, so a later token sees the result of earlier ones. If the pattern has a capture group in brackets, only that group is replaced; the rest of the match is kept.

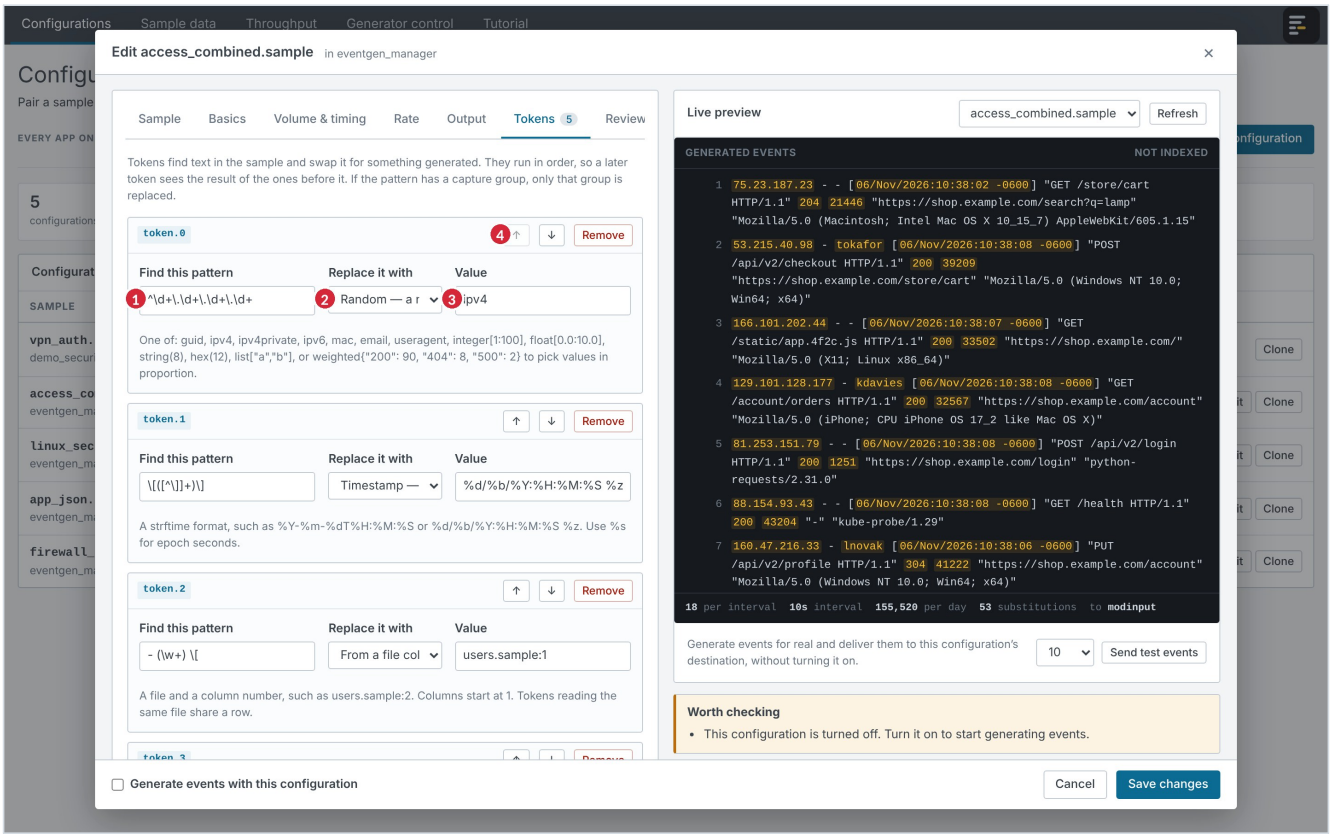


Figure 6.10 — The five tokens of the web access starter.

1	**Find this pattern**: a regular expression. `^\d+\.\d+\.\d+\.\d+` matches the IP at the start of each line.
2	**Replace it with**: the kind of replacement.
3	**Value**: what to generate. The hint under the row lists what is allowed for that kind.
4	Move a token earlier or later, or remove it. **Add token** sits below the last one.

Replace it with	Value examples	Result
Static	`web01.example.com`	Always the same text.
Timestamp	`%Y-%m-%d %H:%M:%S`	A time between earliest and latest, in that format.
Replay timestamp	`%d/%b/%Y:%H:%M:%S`	Replay mode: keeps the original gaps between events.
Random	`ipv4`, `integer[1:100]`, `hex(16)`, `list["a","b"]`	A fresh value each time. Also guid, ipv6, mac, email, useragent, float, string, ipv4private, and weighted.
Rated	`integer[4:1800]`	A random number multiplied by the rate curves.
From a file	`hosts.sample`	A random line from a file.
From a file column	`users.sample:1`	One column of a random line. Tokens reading the same file share a row, so related columns stay consistent.
Counter	`88000`	Counts up on every event. The engine remembers where it got to, so a restart never repeats a number.
From a file, in order	`steps.sample`	The next line each time, starting over at the end.

> **Tip.** Use `weighted{"200": 90, "404": 8, "500": 2}` to pick values in proportion, for example mostly successful status codes with a few errors. This, `ipv4private`, `email`, `useragent` and `uuid` are extensions of the built-in engine; SA-Eventgen does not understand them.

6.7 Review tab

Shows exactly what will be written to `eventgen_manager/local/eventgen.conf` when you save. Use it to double-check, or to copy a stanza into another environment.

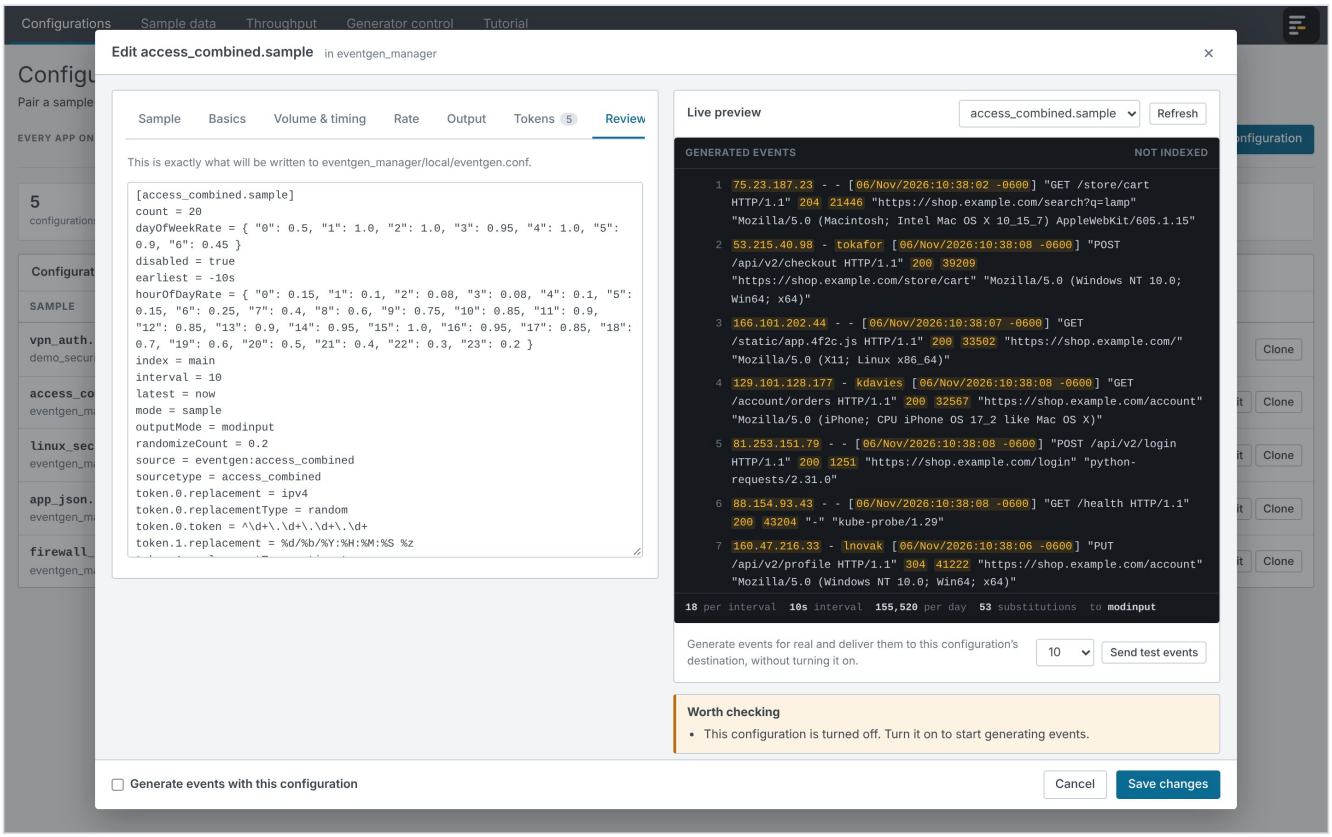


Figure 6.11 — The Review tab shows the stanza as it will be saved.

6.8 Send test events

Under the preview, choose how many events (up to 1,000) and press **Send test events**. The app generates them for real and sends them to the configuration's actual destination, even while the configuration is turned off. A wrong index, a bad HEC token, or a blocked port shows up right away instead of after you start it.

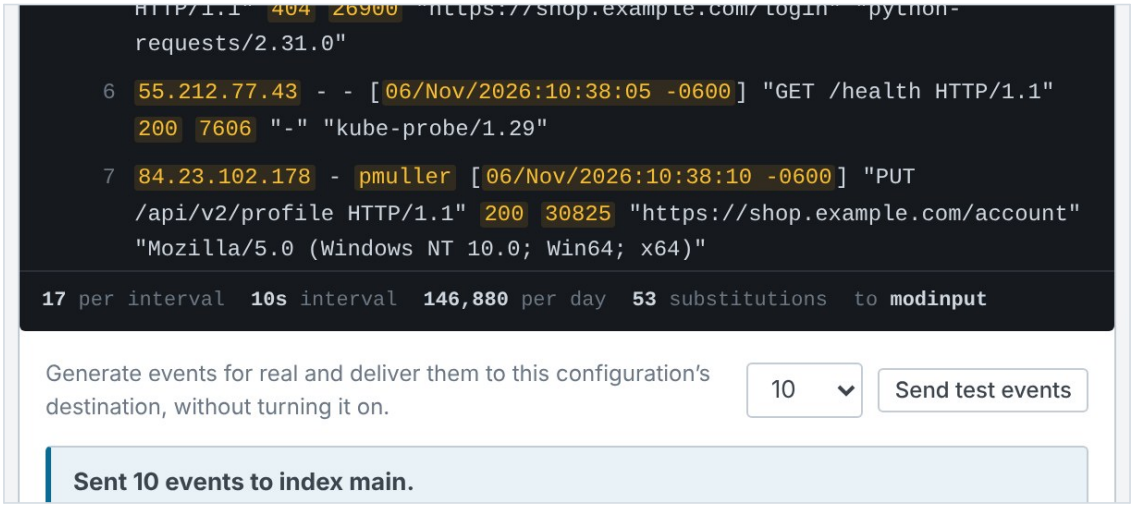


*Figure 6.12 — The result: how many were sent, where, a **Search for them** link, and the first few events.*

6.9 Save

1. Tick **Generate events with this configuration** if you want it running.

2. Press **Save changes** (or **Create configuration**).
3. The engine notices the change at its next check (10 seconds by default) and restarts only that configuration.

7. Build your own data from scratch

This walkthrough creates a brand-new data source: checkout events from a made-up shop API. You will add a sample file, build a configuration for it with six tokens, test it, and start it.

Step 1 — Open Sample data

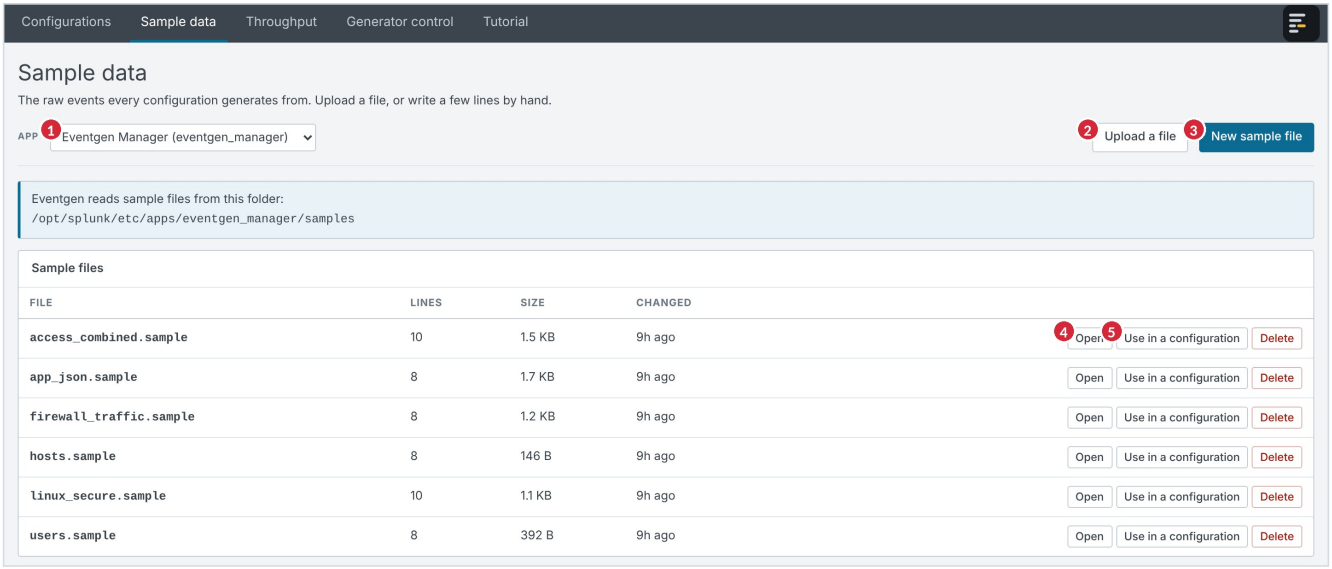


Figure 7.1 — Sample data lists the files in the selected app's samples folder.

1	**App**: which app's samples folder you are looking at.
2	**Upload a file** from your computer (up to 5 MB).
3	**New sample file** lets you type or paste lines.
4	**Open** to view or edit a file.
5	**Use in a configuration** opens the editor for a new configuration.

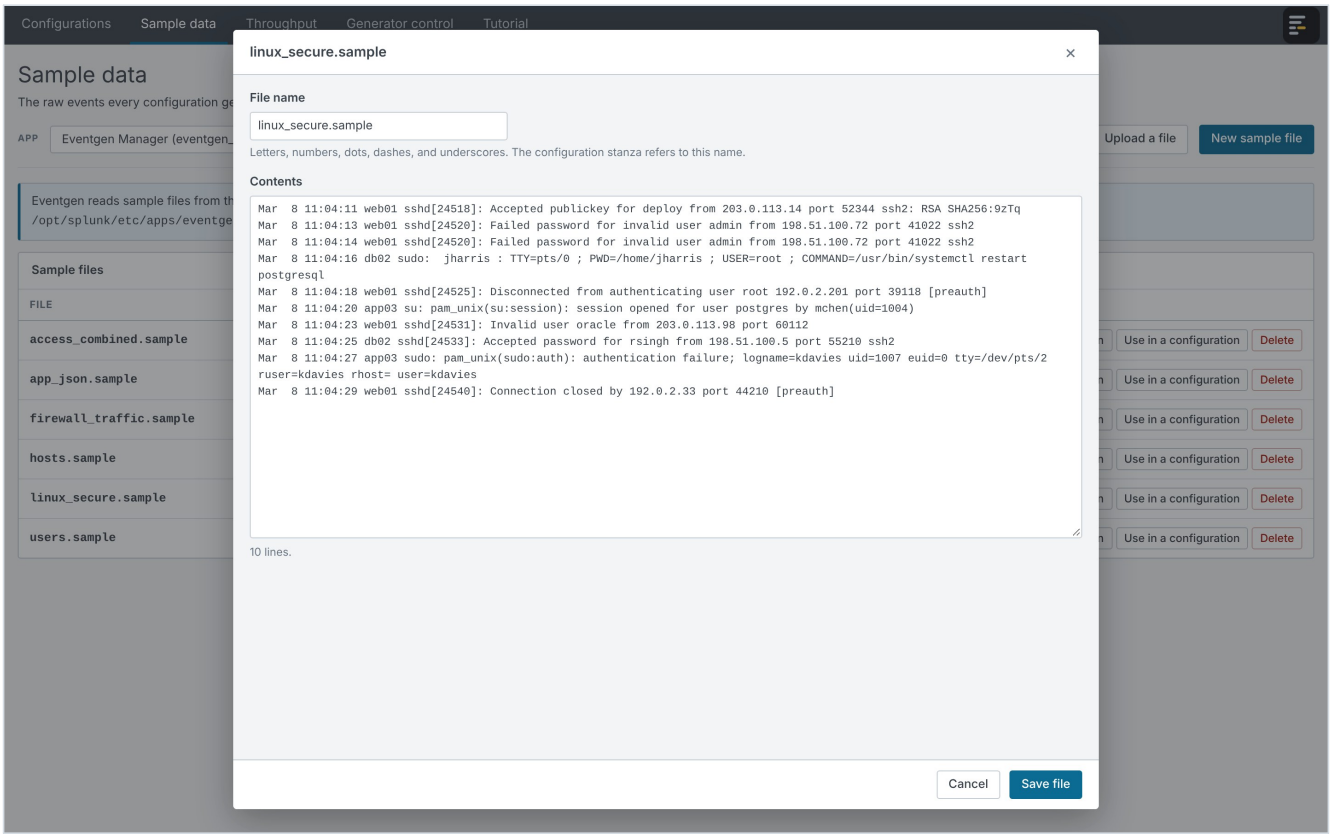


Figure 7.2 — Opening a file shows its contents. You can edit and save small files here.

Step 2 — Create the sample file

1. Press **New sample file**.
2. Enter a **File name**, here `checkout_api.log`. Use letters, numbers, dots, dashes, and underscores.
3. Paste a few example lines into **Contents**. A handful of good lines is enough; tokens will vary them.
4. Press **Create file**.

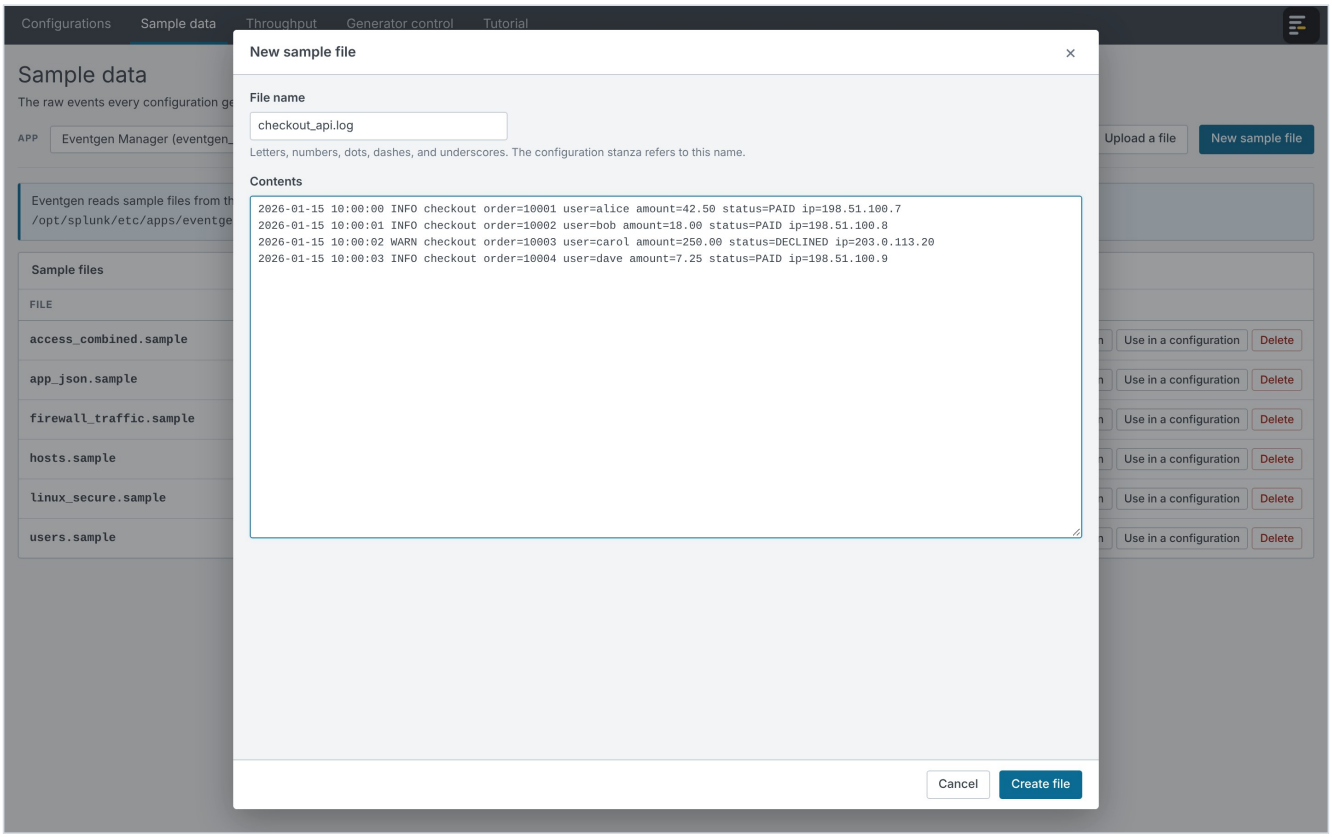


Figure 7.3 — Four example lines are all the sample needs.

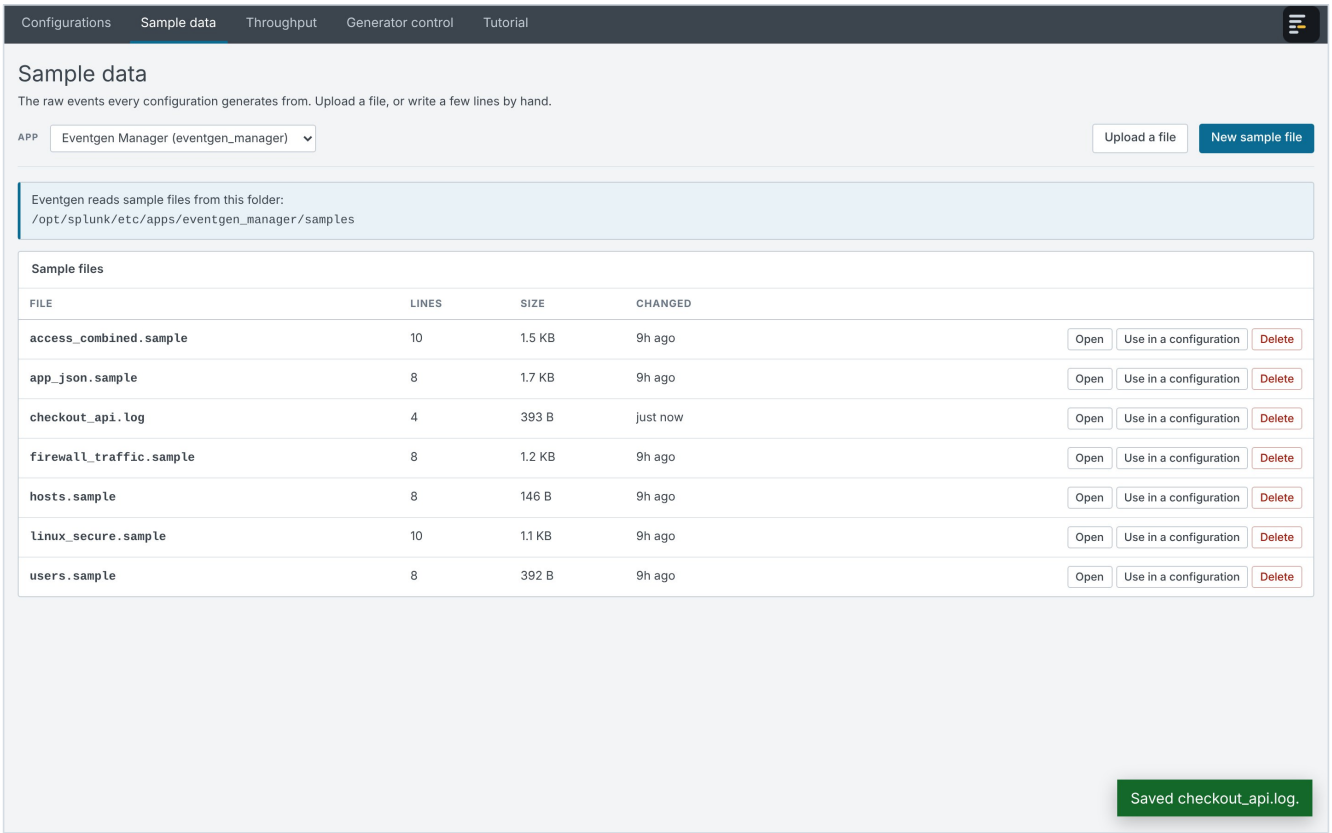


Figure 7.4 — The new file appears in the list.

Step 3 — Start a configuration for it

1. Press **Use in a configuration** on the new file.
2. On the Sample tab, press **Use this** next to `checkout_api.log` so the sample file name is filled in.
3. The preview shows the four lines exactly as written, and **Worth checking** warns that no tokens are set yet.

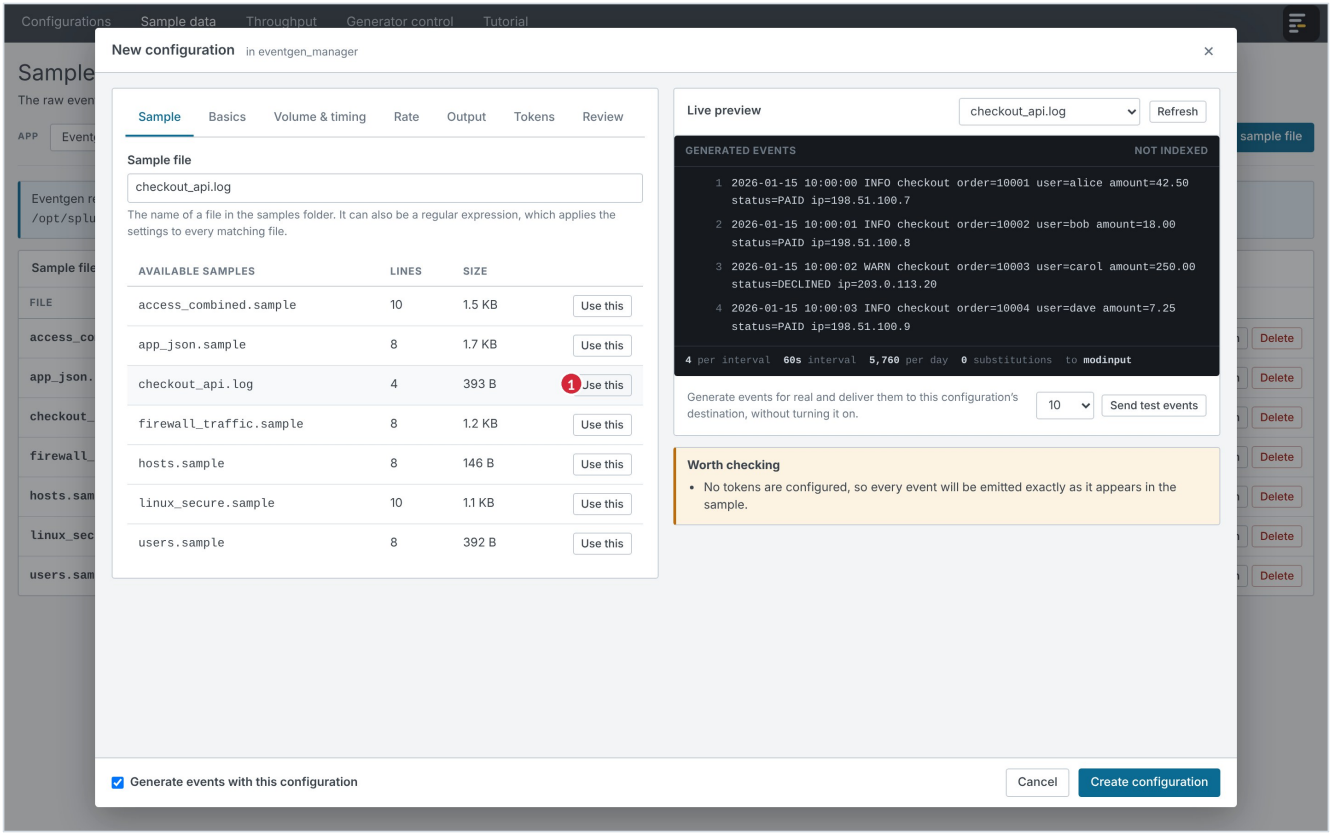


Figure 7.5 — Press Use this (1) to pick the sample for the new configuration.

> **Watch out.** Check the **Sample file** box on the Sample tab before saving. If it is empty (only grey placeholder text), press **Use this** next to your file.

Step 4 — Set volume and timing

On **Volume & timing**, set **Interval** to 20 and **Events per interval** to 8. Set **Earliest event time** to `-20s` and **Latest event time** to `now`, so timestamps fall within each 20-second window. On **Output**, set the sourcetype to `checkout:api` and the source to `eventgen:checkout`.

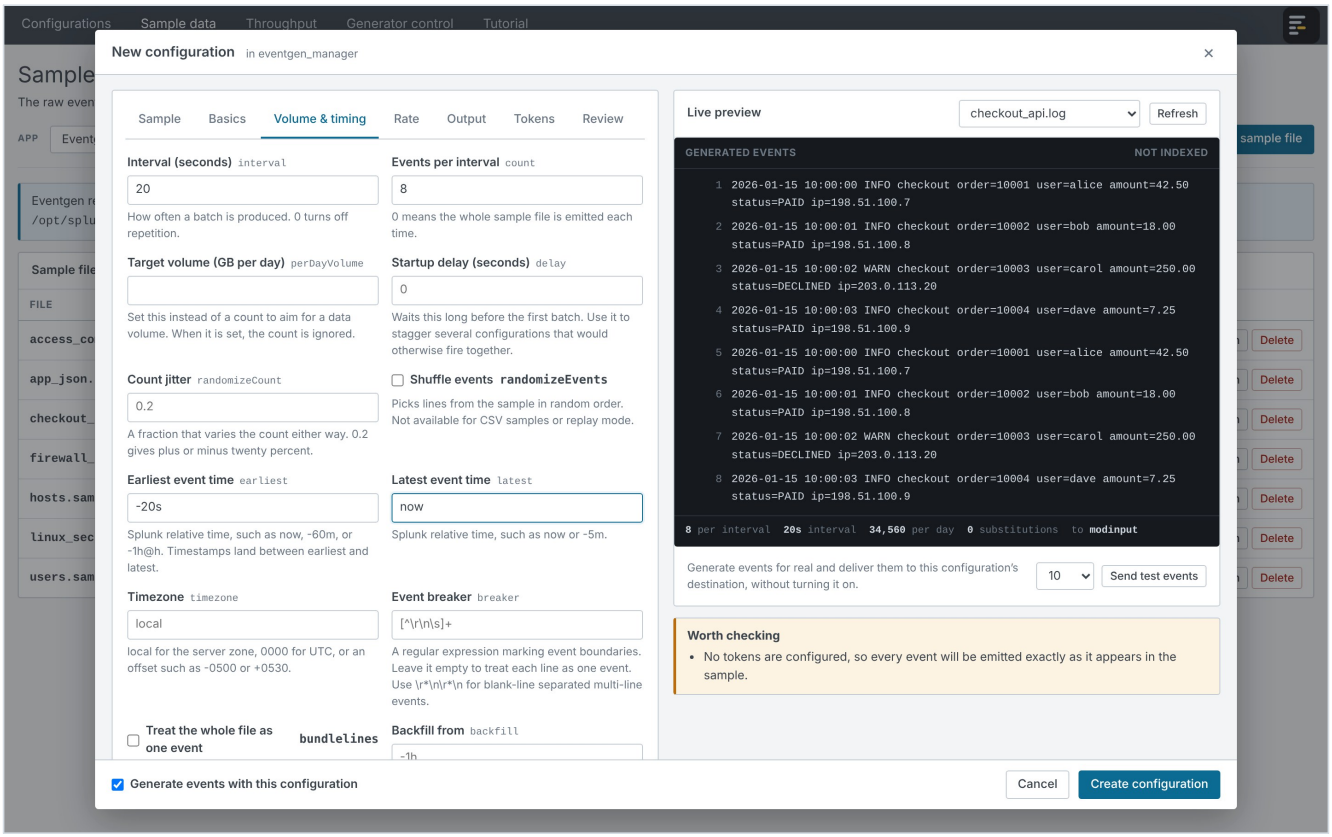


Figure 7.6 — 8 events every 20 seconds. The summary line projects 34,560 events a day.

Step 5 — Add tokens

Open **Tokens**, press **Add the first token**, then **Add token** for each one after. These six make every event different:

#	Find this pattern	Replace it with	Value
0	`^\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}`	Timestamp	`%Y-%m-%d %H:%M:%S`
1	`order=(\d+)`	Counter	`20000`
2	`user=(\w+)`	From a file column	`users.sample:1`
3	`amount=([\d.]+)`	Random	`float[1.00:400.00]`
4	`status=(\w+)`	Random	`weighted{"PAID": 90, "DECLINED": 7, "REFUNDED": 3}`
5	`ip=([\d.]+)`	Random	`ipv4`

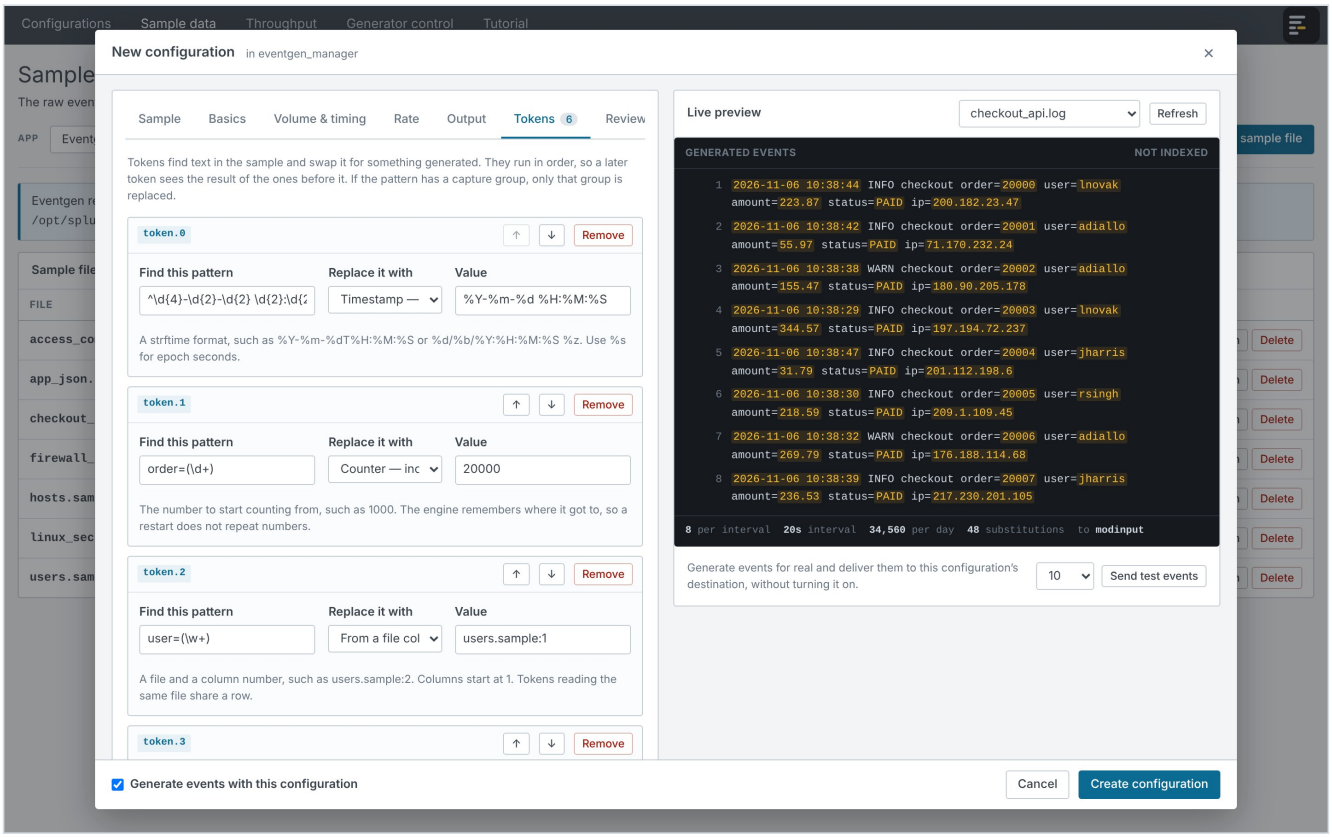


Figure 7.7 — With tokens in place, the preview highlights every generated value: current timestamps, order numbers counting up from 20000, users, amounts, statuses, and IPs.

Notice how token 1 uses a capture group, `order=(\d+)`: the text `order=` stays and only the digits change. Without the brackets the whole match, label included, would be replaced.

Step 6 — Review, test, and create

1. Open **Review** to see the stanza that will be saved.
2. Press **Send test events** and follow **Search for them** to confirm the events reached the index.
3. Make sure **Generate events with this configuration** is ticked.
4. Press **Create configuration**.

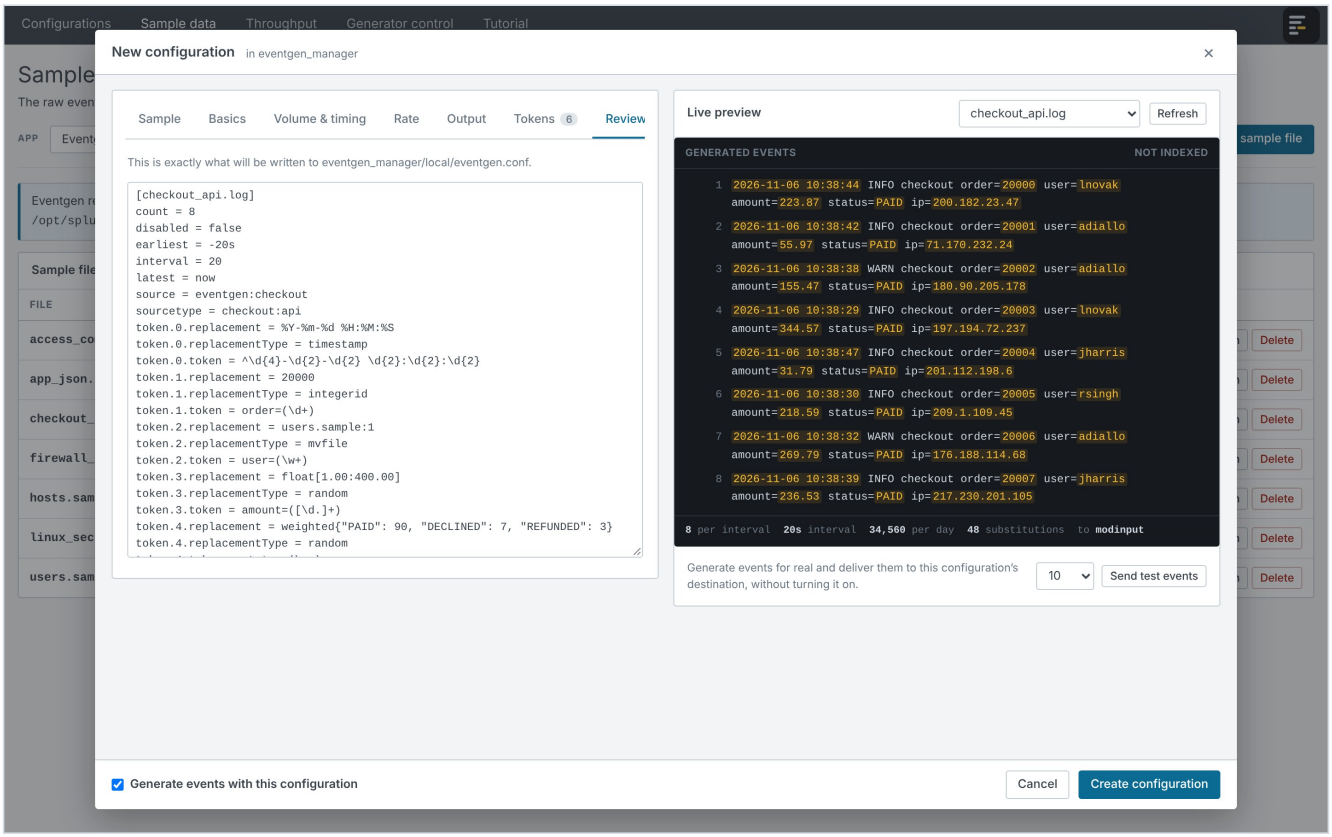


Figure 7.8 — The finished stanza on the Review tab.

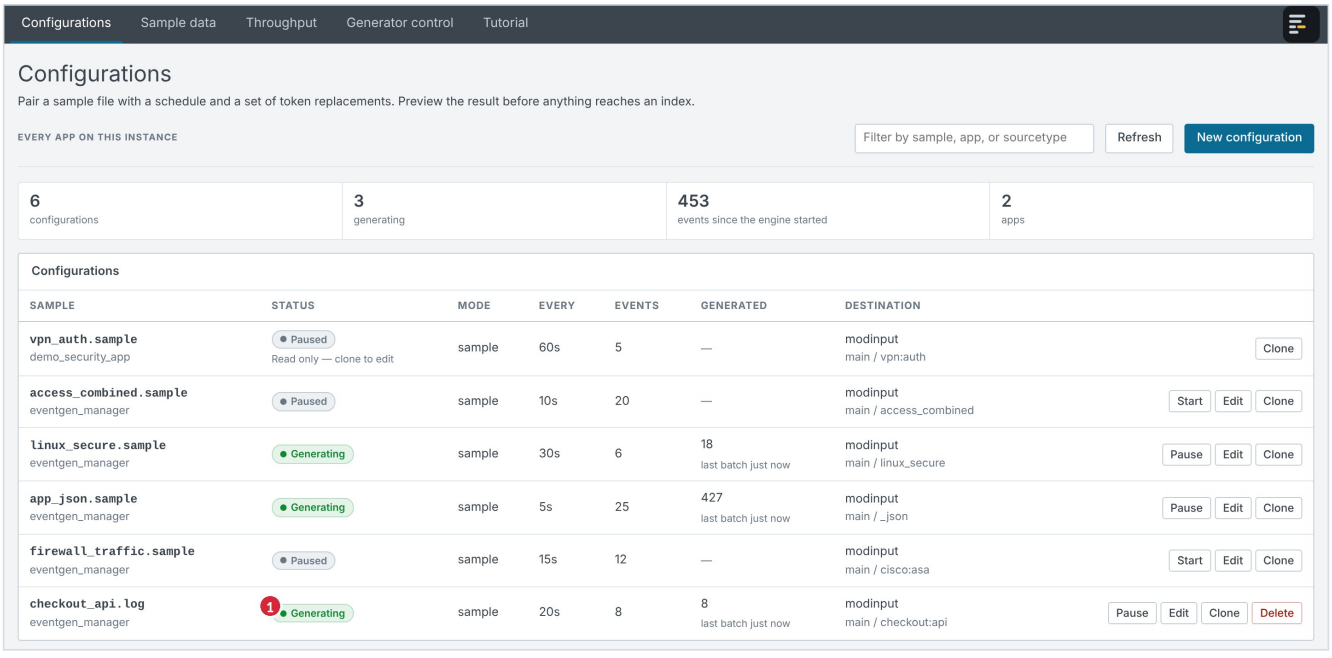


Figure 7.9 — Back on Configurations, the new configuration (1) is generating. It has a Delete button because you created it.

Search for it in Splunk with:

```
index=main sourcetype=checkout:api earliest=-15m
| stats count by status
```

8. Clone a configuration

Cloning copies every setting and token from an existing configuration into a new one in this app. It is the safest way to customise a starter, and the only way to change a configuration that belongs to another app.

1. On **Configurations**, press **Clone** on the row you want to copy.
2. The editor opens as **New configuration** with all settings and tokens filled in, and turned off.
3. On the Sample tab, choose the sample file for the copy (1). It needs a stanza name that does not already exist in this app: a different sample file, or a regular expression that matches the files you want.
4. Adjust anything you like. The tokens came across too (2).
5. Tick **Generate events with this configuration** if you want it running, then press **Create configuration**.

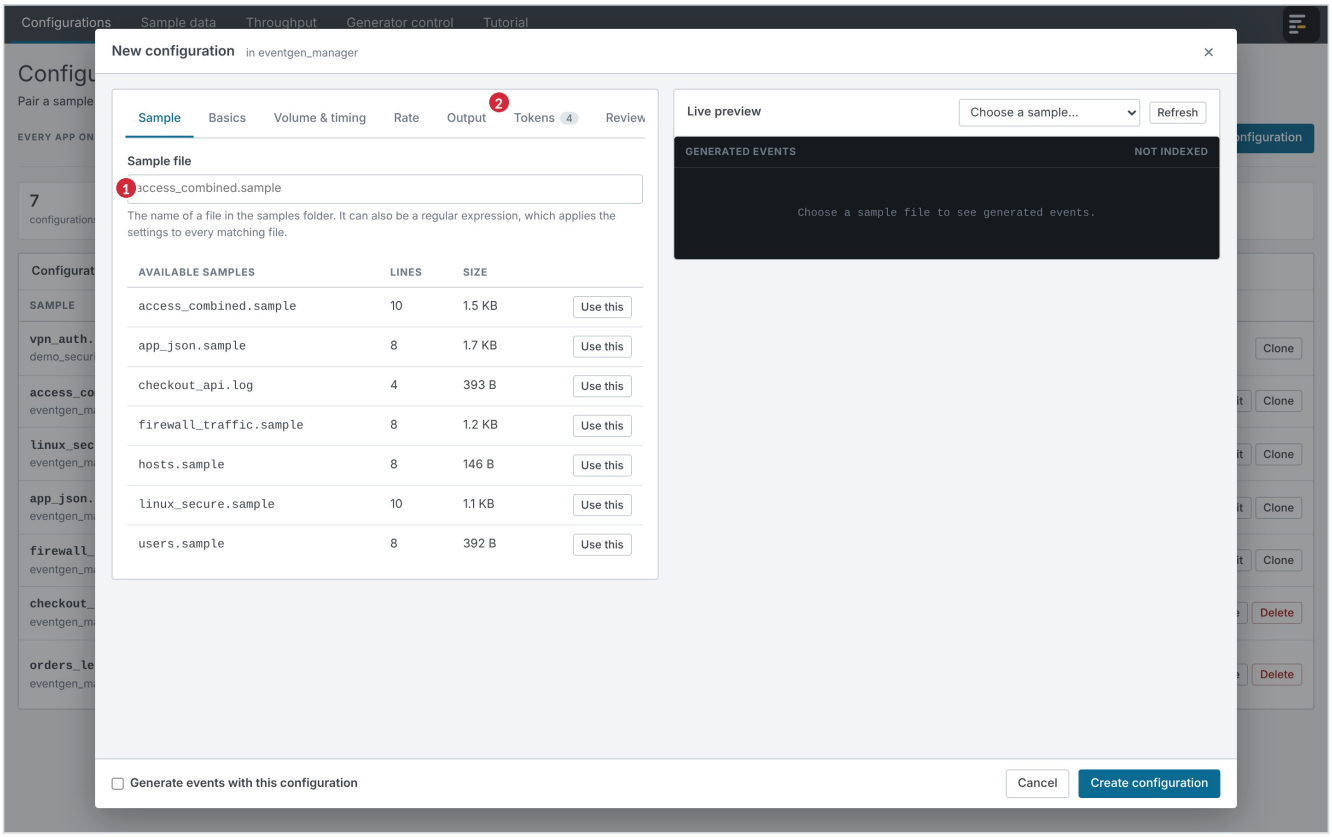


Figure 8.1 — A clone of the firewall starter: four tokens copied, sample name still to choose.

> **Tip.** A good habit: copy a starter's sample to a new name on Sample data (for example `my_web_access.sample`), then clone the starter and pick that file. Your copy lives only in `local/`, so an app upgrade cannot overwrite it. Keep a copy of important sample files outside Splunk as well, or point **Sample directory** at a folder of your own.

Pause, edit, and delete

- **Pause** stops a configuration without losing anything. **Start** resumes it, and counters carry on from where they were.
- **Edit** opens the editor. Changes apply within the reload interval.

- **Delete** removes a configuration you created (you are asked to confirm). The sample file is not touched. Starters cannot be deleted, only paused.

9. Monitor throughput

The **Throughput** page shows what the engine is producing. It reads the metrics line the engine writes for each configuration every 30 seconds to `eventgen_manager_engine.log` in `_internal`.

1. Choose a **Time range** (last 60 minutes by default).
2. Choose a **Configuration**, or leave **All configurations**. Every panel filters to your choice.

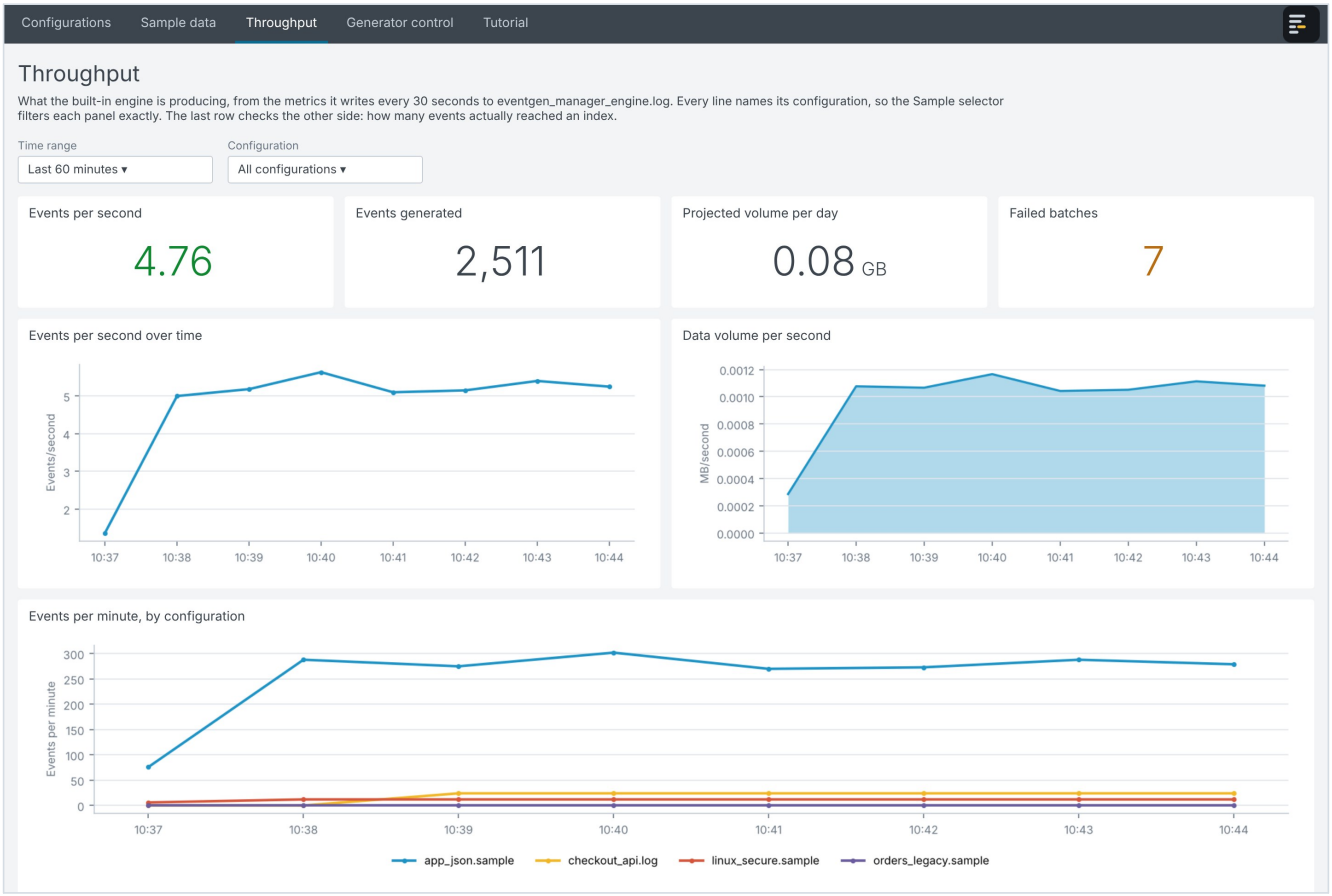


Figure 9.1 — Headline numbers and rate charts. Here four configurations ran for about ten minutes.

Panel	What it tells you
Events per second	Average rate over the time range.
Events generated	Total produced in the time range.
Projected volume per day	Daily volume at the current rate, in GB. Useful for licence planning.
Failed batches	Batches that could not be delivered. Green at 0, amber above 0, red at 10 or more.
Events per second over time / Data volume per second	Trends. Watch for drops when a receiver goes down.
Events per minute, by configuration	Which configuration is producing what.

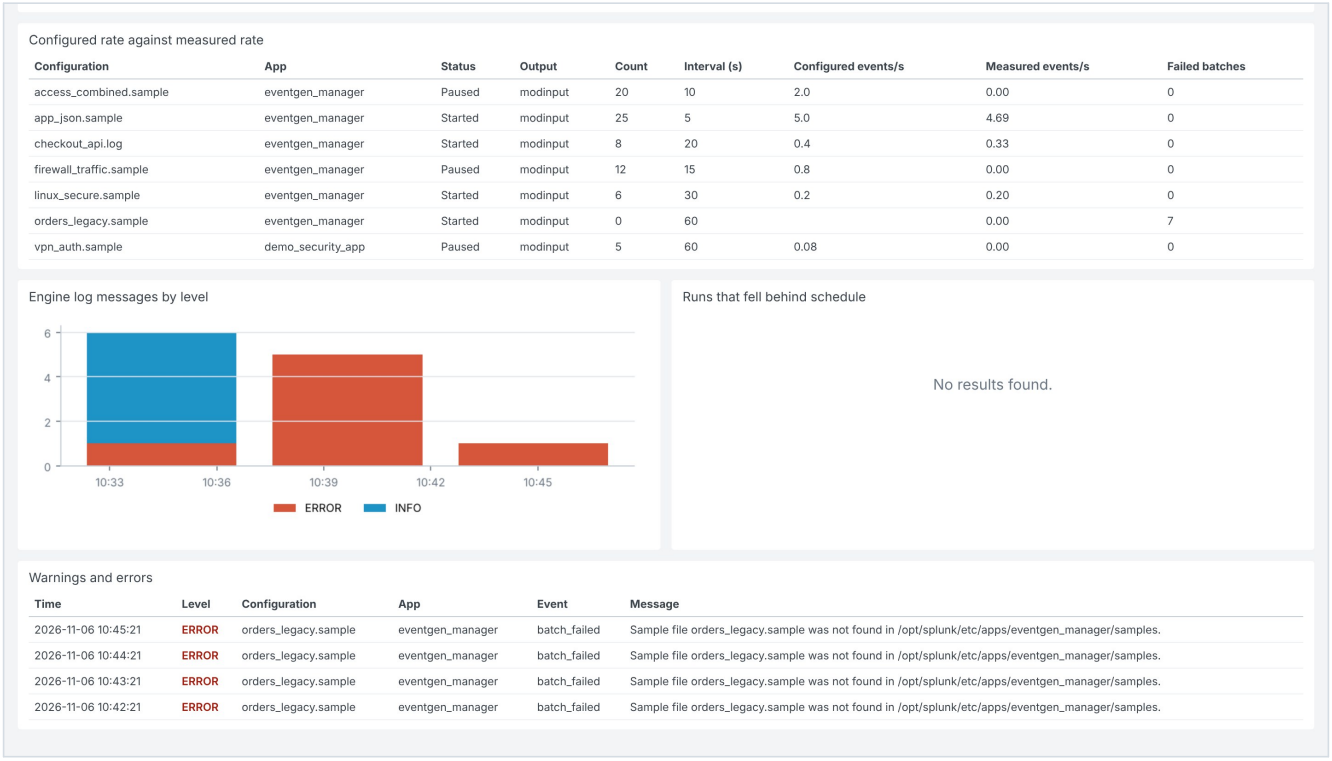


Figure 9.2 — Configured against measured rates, log levels, runs that fell behind, and the warnings and errors table. The failing configuration from Section 11 shows up here.

Panel	What it tells you
Configured rate against measured rate	For each configuration: count, interval, the rate you asked for, and the rate the engine actually achieved. Rate curves and count jitter make measured rates differ from configured ones, and that is expected.
Engine log messages by level	INFO, WARNING, and ERROR messages over time.
Runs that fell behind schedule	Batches that took longer than their interval. If this fills up, reduce the count or tokens.
Warnings and errors	The latest problems with their message. Start here when something is red.
Events that reached an index, by source	The bottom panel (not shown) counts what actually landed in an index, since a wrong index name or a HEC token can drop events after they were sent.

Good to know. If the page is empty, make sure a configuration has run for at least 30 seconds and that your role can search `_internal`.

10. Control the engine

Generator control is also where you change engine settings.

Setting	What it does
Log detail	Debug, Info, Warning, or Error. Written to `eventgen_manager_engine.log`. Use Debug only while troubleshooting.
Check for changes every (seconds)	How quickly a started, paused, or edited configuration takes effect. Default 10, minimum 2.
Stop the engine / Start the engine	Turns all generation off or on at once, without touching any configuration's own on/off setting.
Re-check	Runs the health checks again.

Change a setting, then press **Save settings**. Splunk relaunches the engine by itself to apply it.

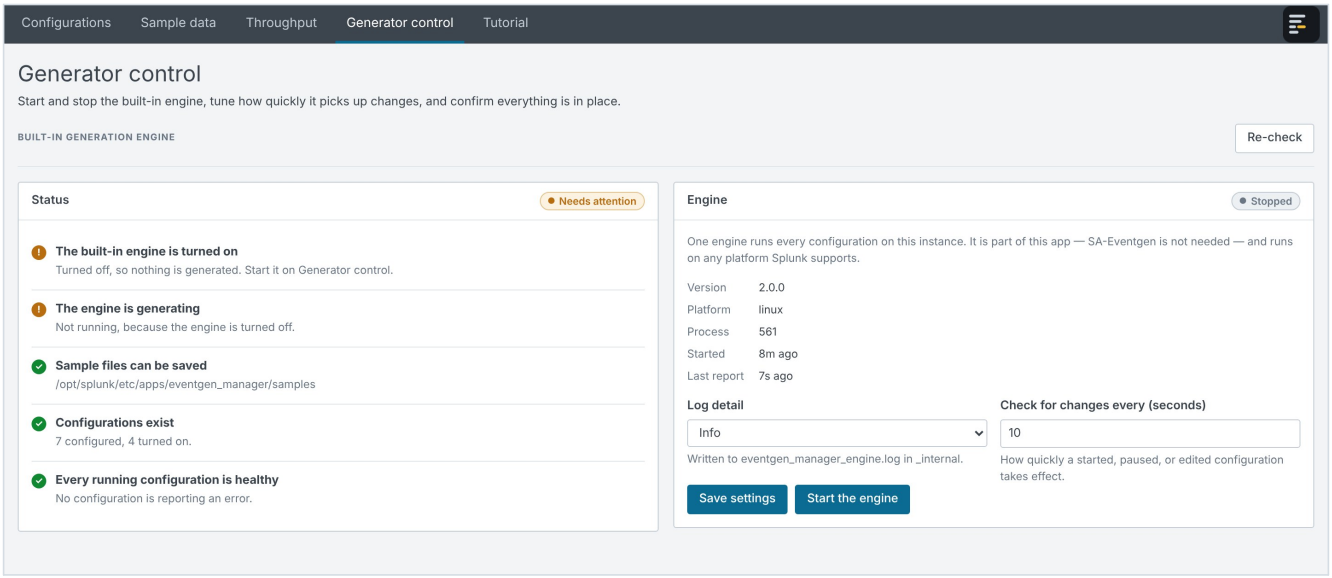


*Figure 10.1 — With the engine stopped, two checks turn amber and the button changes to **Start the engine**.*

Where to run the engine

Run it on one instance per environment: a standalone instance, a heavy forwarder, or a single search head. In a search head cluster every member would run its own engine and duplicate the data, so turn the input off in `local/inputs.conf` on all members but one, or run the app on a separate instance.

A heavy forwarder is a good home for the app: configurations keep the `modinput` output, and the forwarder's `outputs.conf` sends the events on to the indexers over `splunktcp`. See “Sending data to your indexers” in Section 6.5.

What the engine supports

Area	Supported
Modes	sample, replay (paced by replay timestamps or a CSV time field)
Volume	count, perDayVolume, randomizeCount, randomizeEvents, all five rate curves, backfill, delay, end
Samples	raw with any breaker, bundlelines, CSV with per-event host, source, sourcetype, and index
Timestamps	timestamp and replaytimestamp tokens, autotimestamp, and autotimestampTime (generated or now)
Stanza names	file names, or regular expressions matching several files; a [global] stanza sets defaults per app
Outputs	modinput, httpevent, splunkstream, file, spool, tcpout, syslogout (TCP), stdout, devnull
Not supported	s2s output, UDP, custom generator or rater plugins, Jinja templates. Worker and queue tuning settings are accepted and ignored.

As a guide, one configuration produces roughly 25,000 to 55,000 events a second on a single core, depending on how many tokens it has.

11. Read the warning signs

Engine turned off

When the engine is off, the Configurations page says so at the top and every turned-on configuration shows **Idle**. Press **Turn on the engine** (1) and events start within a few seconds.

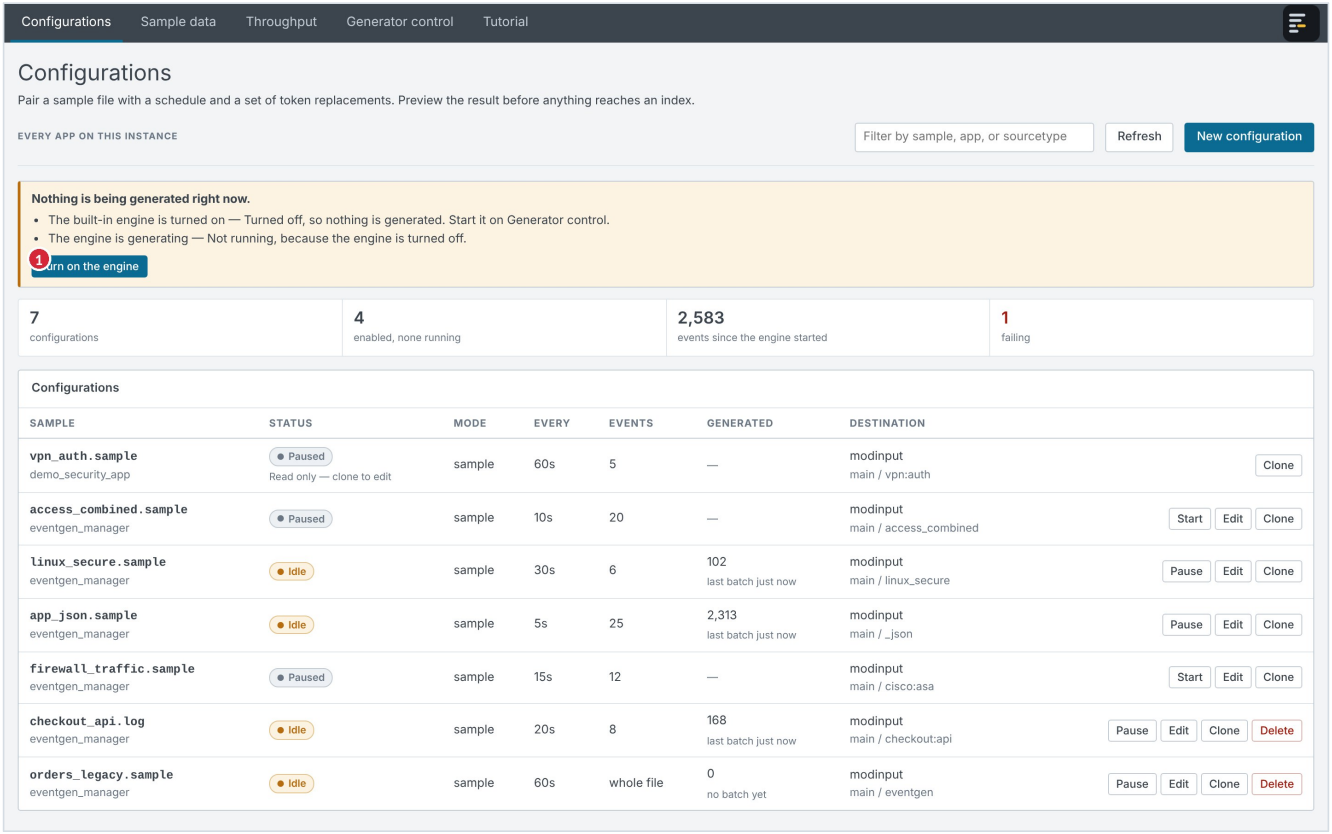


Figure 11.1 — The banner names the failing checks and offers the one button that fixes them.

A configuration in error

Here a configuration was created for `orders_legacy.sample`, a file that does not exist. The preview already warned about it before saving:

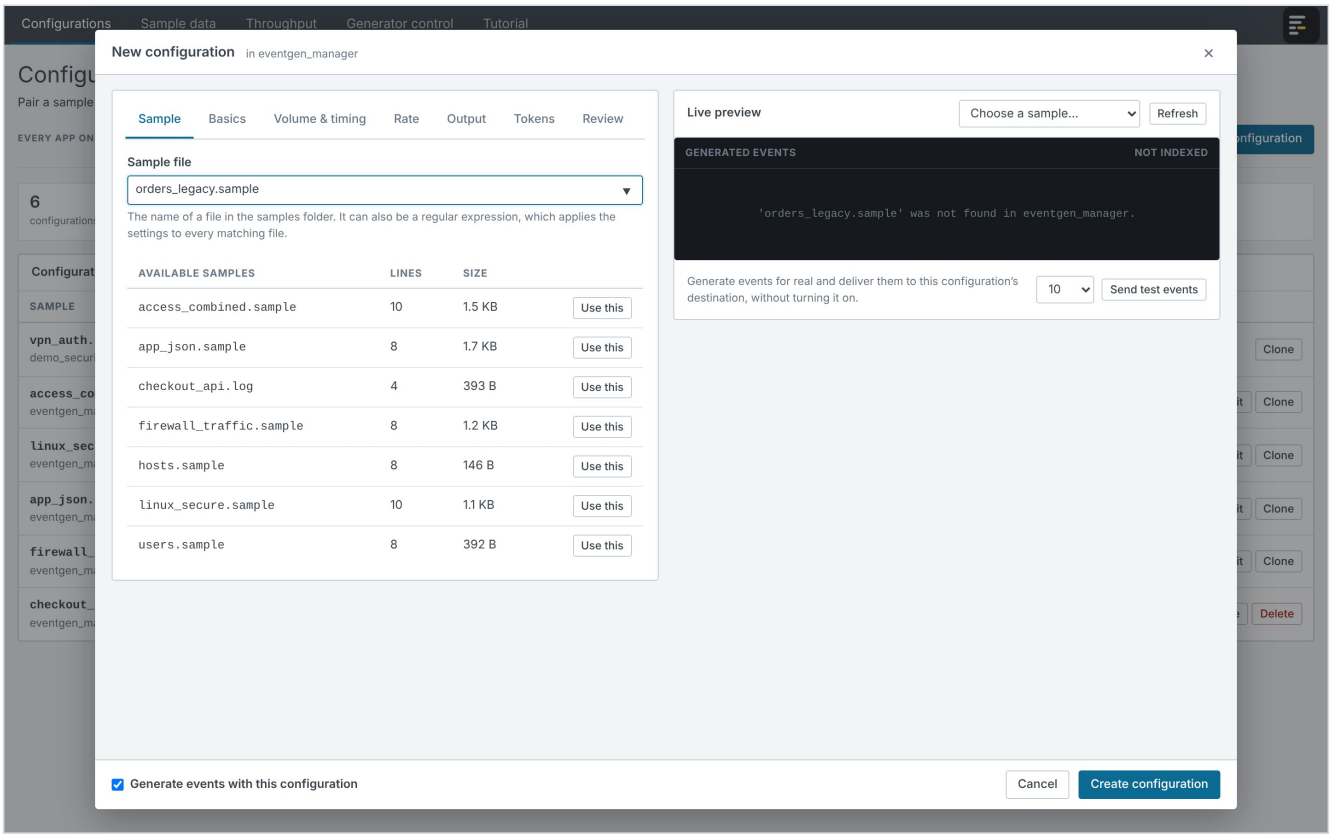


Figure 11.2 — The preview cannot find the file. Fix it here before you create the configuration.

Once saved and turned on, the row shows **Error** with the exact reason (1), and the summary tile turns into a red **failing** count (2). Generator control and the Throughput page report it too.

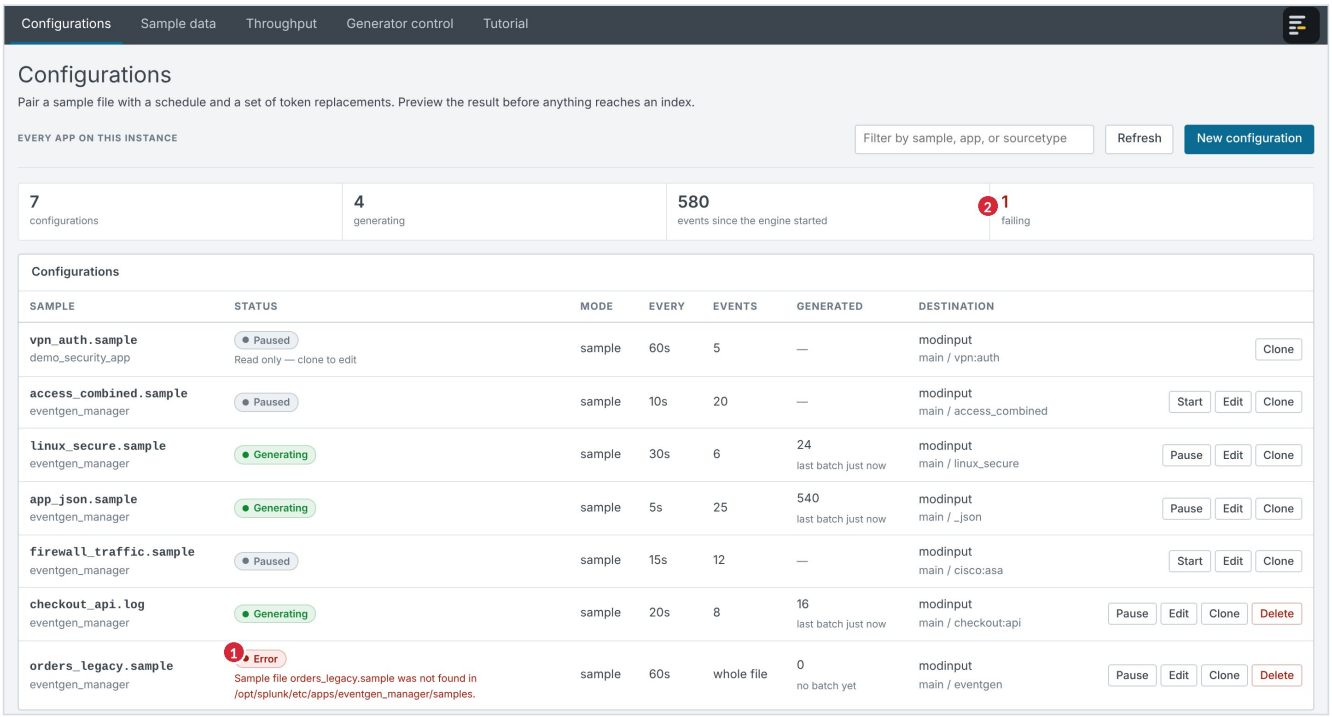


Figure 11.3 — The reason appears under the status badge.

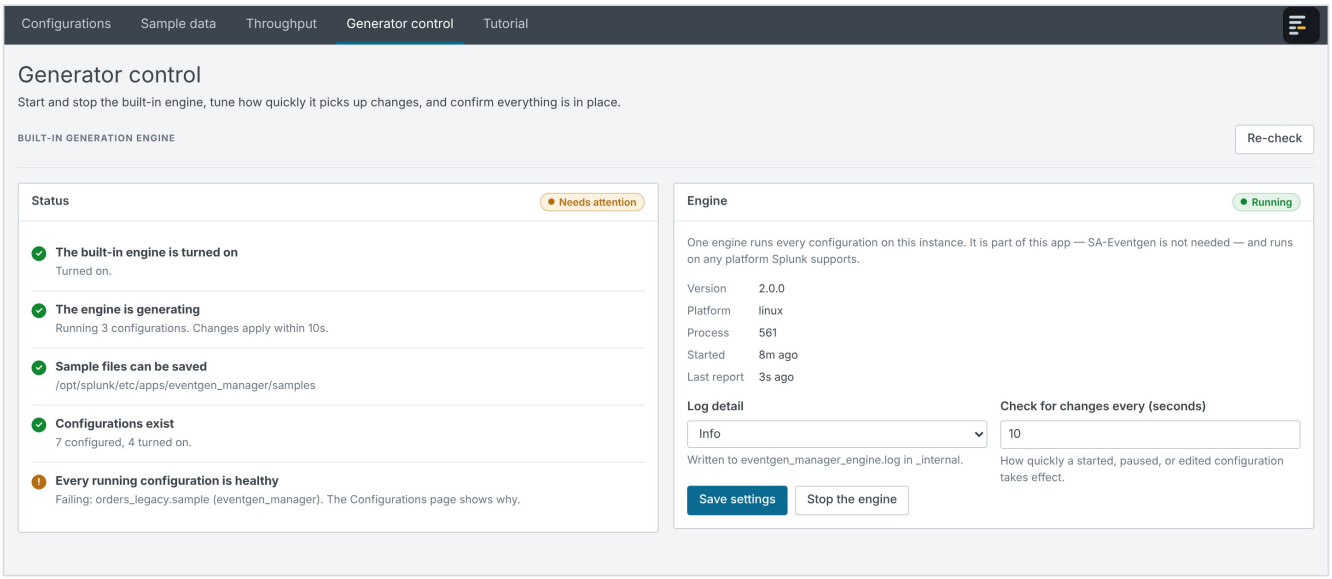


Figure 11.4 — Generator control names the failing configuration.

The engine retries a failing configuration every minute, and right away when you change it or its sample file. In this case, uploading `orders_legacy.sample` on Sample data fixes it without any other step.

12. Troubleshooting

Problem	What to check
Configurations are on but no events appear	Open **Generator control**. It shows whether the engine is on, whether it is reporting, and whether it is on standby because SA-Eventgen is running.
A configuration shows Error	Read the reason under the badge, or the **Warnings and errors** panel on Throughput. The engine retries every minute.
Events have the wrong timestamps	Earliest and latest set the window. Set both to `now` to stamp events at generation time, or, with automatic timestamps, set **Timestamp to write** to **Current time**. A format ending in Z needs `timezone = 0000`.
An alert on the last few minutes never fires	Generated timestamps may fall outside its window. Use earliest and latest of `now`, or Current time for automatic timestamps (Section 6.2.1).
A token does not replace anything	The preview's **Worth checking** box flags tokens that match nothing. Remember a capture group replaces only the first group, and tokens run in order.
HEC output fails with a certificate error	Splunk's default HEC certificate is self-signed. Install a trusted certificate, or untick **Verify the receiver's certificate** for a test receiver.
Permission errors when saving	Creating and changing configurations needs the admin or sc_admin role.
Nothing on the Throughput page	Let a configuration run for 30 seconds, and check your role can search `index=_internal source=*eventgen_manager_engine.log*`.
Starters stopped after a reinstall	Deleting the app folder removed `local/`. Start them again, or use clones, which live only in `local/`.
Events do not reach the indexers	Check `outputs.conf` on the instance running the app, that the index exists on the indexers, and any `_TCP_ROUTING` group name. Raw TCP cannot send to a `splunktcp` port.
Test events sent, but search finds nothing	Check the index exists and your role can write to it. Indexing can take a few seconds.

Useful searches

```
index=_internal source=*eventgen_manager_engine.log* level IN (WARNING, ERROR)

index=_internal source=*eventgen_manager_engine.log* event=metrics
| timechart span=1m sum(events) by stanza
```

13. Reference

Coming from SA-Eventgen

Your existing configurations need no changes; the engine reads the same `eventgen.conf` in the same apps. If SA-Eventgen's input is still running, the built-in engine waits on **Standby** so events are not doubled. On Generator control, press **Switch to the built-in engine**; it turns SA-Eventgen's input off and the engine takes over within seconds. You can uninstall SA-Eventgen afterwards if nothing else needs it.

Three behaviours differ from SA-Eventgen:

- The **rated** replacement works (SA-Eventgen 8.2.1 rejected it).
- In sample mode, each interval carries on through the sample file where the last one stopped, so every line gets used over time.
- Syslog goes over TCP, and `udpout` is not available.

Where things are stored

What	Where
Configurations you create or edit	`etc/apps/eventgen_manager/local/eventgen.conf`
Sample files	`etc/apps/eventgen_manager/samples/`
Starter configurations	`etc/apps/eventgen_manager/default/eventgen.conf`
Engine settings	`etc/apps/eventgen_manager/local/inputs.conf`
Engine status and counters	`var/lib/splunk/modinputs/eventgen_manager_engine/status.json`
Engine log	`var/log/splunk/eventgen_manager_engine.log`

Configurations are read and written through Splunk's own `configs/conf-eventgen` endpoint, so permissions, configuration layering, and search head cluster replication work as they do for any other Splunk setting.

Run the engine outside Splunk

The engine also runs from the command line, which is handy for checking a configuration or producing a file of test data on a laptop. Configurations whose output is `modinput` print to standard output in this mode.

```
python bin/eventgen_manager_engine.py run \
    --conf default/eventgen.conf --include-disabled --duration 30

# only some configurations
python bin/eventgen_manager_engine.py run \
    --conf default/eventgen.conf --include-disabled \
    --stanza linux_secure.sample --duration 10
```

Quick checklist

1. Generator control is all green.
2. Your sample file is on Sample data, with a few realistic lines.

3. The configuration's preview looks right and Worth checking is clear.
4. Send test events reached the index you expect.
5. Generate events is ticked and saved; status shows Generating.
6. Throughput shows the rate you planned, with no failed batches.